**Department of Computer and Software Engineering**

**College of E&ME, NUST, Rawalpindi**

# EC-240 Data Base Engineering

# Final Project Report

**Course Instructor: Dr Shahzad**

**CE 45 A**

| Sr. No. | Student's Name | CMSID |
|---|---|---|
| 1 | Rida Zahra | 465791 |
| 2 | Tazeen ur Rehman | 466342 |
| 3 | Mawa Chaudhary | 474121 |
| 4 | Muhammad Hassan Ali | 456543 |

**Date : 27th April, 2026**

# Table of Contents

# 1. Project Overview

MarketHub is a fully relational, data-driven online marketplace system modeled after platforms such as Amazon and Daraz. It enables buyers to register, browse products, manage a cart, place orders, make payments, and submit reviews. Sellers can list products across hierarchical categories, manage inventory, and track their sales through analytical views. The system is implemented in Microsoft SQL Server (T-SQL) and exposes a web-based frontend interface.

## 1.1 Project Title

**"Online Marketplace System - MarketHub" (Amazon / Daraz-like E-Commerce Platform)**

## 1.2 Objectives

- Design and implement a complete, normalized relational database system for a realistic e-commerce domain.
- Apply advanced ER modeling concepts: specialization/generalization, weak entities, multivalued attributes, and composite attributes.
- Normalize all 21 tables to Third Normal Form (3NF) with full justification.
- Implement a rich SQL layer covering DDL, DML, analytical queries, joins, subqueries, GROUP BY, HAVING, indexes, views, triggers, and user-defined functions.
- Develop a simple frontend interface that maps user actions to underlying SQL operations.
- Demonstrate practical database engineering skills aligned with CLO3 → PLO3 of EC-240.

## 1.3 Technology Stack

| Component | Technology / Tool |
|---|---|
| Database Engine | Microsoft SQL Server (T-SQL) |
| Schema Design Tool | DrawSQL / ERDPlus (conceptual & relational diagrams) |
| SQL Client | SQL Server Management Studio (SSMS) / Azure Data Studio |
| Backend | Python (Flask) / PHP |
| Frontend | HTML5, CSS3, JavaScript (Vanilla / Bootstrap) |
| Version Control | Git / GitHub (team collaboration) |

# 2. Intended Users of the System

The system is designed to serve three categories of users. The ISA specialization in the ER model (complete, disjoint) captures the formal distinction between the two persisted roles.

| User Role | ER Entity | Capabilities |
|---|---|---|
| Buyer | Buyer (subtype of User) | Browse products, manage cart, place orders, make payments, write reviews, track order history, earn loyalty points. |
| Seller | Seller (subtype of User) | Register a shop, add/update/soft-delete product listings, manage inventory across warehouses, view sales analytics, receive payments. |
| Guest / Public | Not stored in DB | Browse the product catalogue only. Must register to place orders or write reviews. |
| Admin (Future) | Permission flag on User | Monitor system activity, manage categories, view platform-wide reports. Removed from current design to stay within scope. |

Note: The ISA constraint is Set{complete, disjoint} - every registered User must be exactly one of Buyer or Seller, and no User may belong to both subtypes simultaneously.

# 3. Functional Requirements

The following 15 functional requirements were identified through domain analysis of a real-world e-commerce platform. They are grouped by subsystem and mapped to their corresponding ER entities.

## 3.1 User Management

- FR-01: Users can register an account with a unique email address and a securely hashed password.
- FR-02: Users can log in and log out of the system securely using their email and password.
- FR-03: The system supports two user subtypes via ISA specialization - Buyer and Seller - with a complete, disjoint constraint.
- FR-04: Users can store, update, and manage multiple delivery and billing addresses, designating one as the default.

## 3.2 Product & Catalog Management

- FR-05: Sellers can add, update, and soft-delete (status = 'deleted') product listings, including multiple images (ProductImage) and dynamic key-value attributes (ProductAttribute).
- FR-06: Products are organized in a two-level hierarchy: Category → Subcategory → Product, enabling structured navigation.
- FR-07: Buyers can browse, filter, and search products by category, subcategory, price range, and average rating.
- FR-08: Product stock is maintained at the Product level. A TRIGGER automatically decrements stock upon order placement and prevents stock from dropping below zero.

## 3.3 Shopping Cart

- FR-09: Each registered Buyer has exactly one persistent shopping Cart
- FR-10: Buyers can add, remove, and update quantities of products in their cart. CartItem resolves the M:N between Cart and Product.

## 3.4 Order Processing

- FR-11: Buyers can place orders containing multiple products in a single transaction. OrderItem resolves the M:N between Order and Product, storing unit_price as a historical snapshot.
- FR-12: An order must reference a valid delivery address belonging to the ordering buyer and is directly linked to a Shipment record.

## 3.5 Payment & Transactions

- FR-13: Each order triggers exactly one Payment record . Each payment can generate multiple Transaction records to support retry logic and audit trails.

## 3.6 Inventory & Shipping

- FR-14: Products can be stored across multiple Warehouses. The Inventory entity resolves the relationship between Product and Warehouse, tracking quantity and a low-stock threshold per location.
- FR-15: Each order is linked to exactly one Shipment, which references a Carrier and tracks dispatch status and tracking number.

## 3.7 Reviews & Analytics

- FR-16: Buyers can submit exactly one review per product (rating 1–5 + optional comment).
- FR-17: The system supports analytical reporting: top-selling products, most active buyers, monthly revenue trends, inventory levels, and seller dashboards. [SQL Views + Analytical Queries]

# 4. Business Rules & Constraints

All business rules below are enforced either at the database level (via PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, TRIGGER, or DEFAULT constraints) or at the application level. The enforcement method is noted for each rule.

## 4.1 User Rules

- BR-01: Every User must have a unique email address. [UNIQUE constraint on User.email]
- BR-02: Every User must have a unique, auto-generated user_id. [PK IDENTITY(1,1)]
- BR-03: A User is exactly one of Buyer or Seller - not both and not neither. [Complete, disjoint ISA - enforced by application-level registration logic]
- BR-04: A Buyer's date_of_birth must indicate they are at least 16 years old at time of registration. [CHECK: DATEDIFF(YEAR, date_of_birth, GETDATE()) >= 16]
- BR-05: A User can have zero or more Addresses; each Address belongs to exactly one User. [1:M FK, CASCADE DELETE]
- BR-06: Seller.rating is a derived attribute equal to AVG(Review.rating) across all reviews on their product listings. [Computed via fn_SellerRating() and View]

## 4.2 Product Rules

- BR-07: Every Product must belong to exactly one Seller (seller_id FK, NOT NULL).
- BR-08: Every Product must belong to exactly one Subcategory (subcategory_id FK, NOT NULL).
- BR-09: Product.price must be strictly greater than zero. [CHECK price > 0]
- BR-10: Product.stock cannot go below zero. [CHECK stock >= 0; enforced also by Trigger trg_prevent_negative_stock]
- BR-11: When an OrderItem is inserted, stock is decremented by OrderItem.quantity automatically. [TRIGGER: trg_reduce_stock]
- BR-12: Product deletion is a soft delete - status is set to 'deleted' and the record is retained. [CHECK status IN ('active','inactive','deleted')]

## 4.3 Cart Rules

- BR-13: Each Buyer has exactly one Cart at any time. [UNIQUE constraint on Cart.buyer_id]
- BR-14: CartItem.quantity must be >= 1. [CHECK quantity >= 1]
- BR-15: The same product cannot appear twice in the same cart row. [Composite PK: (cart_id, product_id)]

## 4.4 Order Rules

- BR-16: An order must reference a valid buyer and a valid delivery address belonging to that buyer.
- BR-17: An order directly references its Shipment via shipment_id FK. [UNIQUE on Order.shipment_id - 1:1]
- BR-18: OrderItem.unit_price stores the price at purchase time as a historical snapshot.
- BR-19: Order.total_amount is a derived attribute: SUM(OrderItem.unit_price × OrderItem.quantity). [Computed via fn_CalculateOrderTotal() and TRIGGER]
- BR-20: Order.status values are restricted to: 'pending', 'confirmed', 'shipped', 'delivered', 'cancelled', 'returned'. [CHECK / ENUM]

## 4.5 Payment Rules

- BR-21: Each Order has exactly one Payment record. [UNIQUE constraint on Payment.order_id]
- BR-22: Payment.amount must be > 0. [CHECK amount > 0]
- BR-23: Payment.status values are restricted to: 'pending', 'completed', 'failed', 'refunded'. [CHECK]
- BR-24: Each Payment can generate one or more Transaction records (1:M) to support retry logic and full audit trail.

## 4.6 Inventory Rules

- BR-25: Each (product_id, warehouse_id) pair must be unique in Inventory. [UNIQUE(product_id, warehouse_id)]
- BR-26: Inventory.quantity >= 0 and Inventory.min_threshold >= 0. [CHECK constraints]

## 4.7 Review Rules

- BR-27: A Buyer may submit only one review per product. [UNIQUE(buyer_id, product_id) on Review]
- BR-28: Review.rating must be between 1 and 5 inclusive. [CHECK rating BETWEEN 1 AND 5]
- BR-29: A buyer should only review a product they have ordered and received. [Application-level enforcement]

# 5. Conceptual Design - ER Model

The Entity-Relationship (ER) model was designed to accurately represent the complexity of the MarketHub e-commerce domain. All required advanced modeling concepts have been incorporated.

## 5.1 Entity Inventory

| Entity | Type | Description |
|---|---|---|
| User | Strong | Central entity for all registered accounts. Has user_id (PK), name, email (UNIQUE), password_hash, phone, created_at. |
| Buyer | Subtype (ISA) | Specialization of User. Adds loyalty_points, preferred_payment, date_of_birth. |
| Seller | Subtype (ISA) | Specialization of User. Adds shop_name, description, bank_account, rating (derived). |
| Address | Strong | Stores multiple addresses per User (1:M). Composite: street, city, country, zip_code. Has is_default flag. |
| Category | Strong | Top-level product classification (e.g. Electronics, Clothing). |
| Subcategory | Strong | Second-level classification nested under Category. |
| Product | Strong | Core catalog entity. Has price (CHECK > 0), stock (CHECK >= 0), status (soft-delete). |
| ProductImage | Weak | Zero or more images per product. Depends on Product via CASCADE DELETE. |
| ProductAttribute | Weak | Dynamic key-value specs per product (color, size, material). Depends on Product. |
| Cart | Strong | One persistent cart per Buyer (1:1). |
| CartItem | Weak / Junction | Resolves M:N between Cart and Product. Composite PK: (cart_id, product_id). Stores quantity. |
| Inventory | Junction | Resolves M:N between Product and Warehouse. Stores quantity per location with min_threshold. |
| Warehouse | Strong | Physical storage location with name, location, capacity. |
| Carrier | Strong | Shipping courier with tracking URL and contact details. |
| Shipment | Strong | Tracks dispatch status and tracking number for an order. |
| Order | Strong | Central transactional entity. References Buyer, Address, Shipment (1:1). total_amount is derived. |
| OrderItem | Weak / Junction | Resolves M:N between Order and Product. Composite PK: (order_id, product_id). Stores unit_price snapshot. |
| Payment | Strong | One payment per order (1:1). Tracks method, status, amount, paid_at. |
| Transaction | Strong | One or more gateway transaction records per Payment (1:M). Audit trail. |
| Review | Strong | One review per buyer per product. Stores rating (1-5) and comment. |

## 5.2 Advanced Modeling Concepts

### 5.2.1 Specialization / Generalization

The User entity is specialized into Buyer and Seller using a complete, disjoint ISA hierarchy. Complete means every User must be either a Buyer or Seller (no plain User can exist). Disjoint means a User cannot simultaneously hold both roles.

### 5.2.2 Weak Entities

ProductImage and ProductAttribute are weak entities - they cannot exist without their parent Product (CASCADE DELETE). CartItem and OrderItem use composite primary keys and are considered weak entities relative to their parent entities.

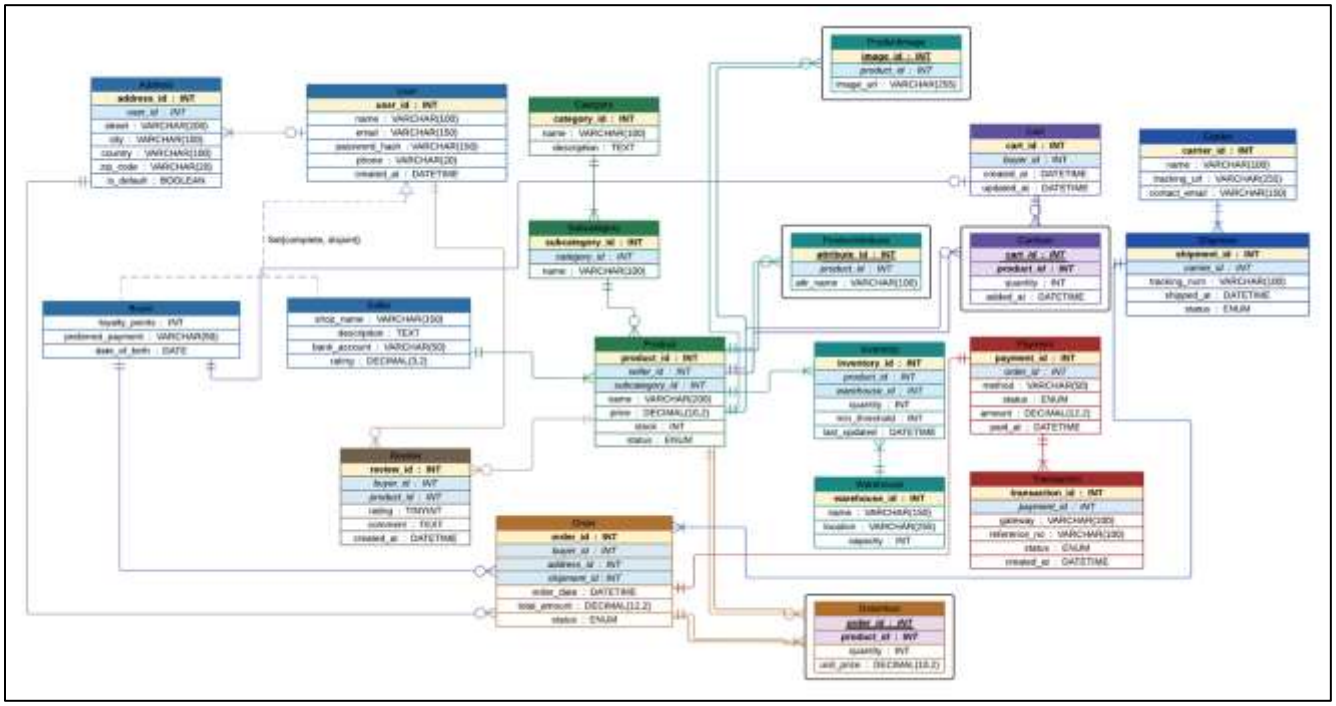
### 5.2.3 Many-to-Many Relationships

- Order ↔ Product → resolved by OrderItem (composite PK: order_id + product_id)
- Cart ↔ Product → resolved by CartItem (composite PK: cart_id + product_id)
- Product ↔ Warehouse → resolved by Inventory (product_id + warehouse_id, UNIQUE)
- Buyer ↔ Order → resolved by Buyer_Order (supports shared/group orders)

### 5.2.4 Multivalued, Composite & Derived Attributes

- Multivalued: Address (a User can have many), ProductImage, ProductAttribute
- Composite: Address is decomposed into street, city, country, zip_code - each stored atomically
- Derived: Order.total_amount = SUM(OrderItem.unit_price × quantity); Seller.rating = AVG(Review.rating)

## 5.3 ER Diagram



| Requirement Aspect | Expected | Implemented in ER Diagram |
|---|---|---|
| Number of Entities | 6–8 entities | 12+ entities (User, Buyer, Seller, Product, Category, Subcategory, Cart, Order, Payment, Shipment, Inventory, Warehouse, Review, etc.) |
| Many-to-Many Relationships | At least 2 | 4 M:N relationships resolved via associative entities: • Order ↔ Product (OrderItem) • Cart ↔ Product (CartItem) • Product ↔ Attribute (ProductAttribute) • Buyer ↔ Product (Review) |
| Specialization / Generalization | At least 1 | User specialized into Buyer and Seller |
| Weak / Associative Entities | Use where applicable | CartItem, OrderItem, Review act as associative (weak-like) entities |
| Attribute Variety | Basic + advanced attributes | Includes different types: • Simple (name, price) • Derived/controlled (status ENUM) • Multi-record (ProductImage) |
| Relationship Cardinality | Clearly defined | 1:N and M:N relationships properly represented (e.g., User–Address, Seller–Product, Order–Shipment) |
| Data Integrity Consideration | Logical consistency | Use of keys (IDs), structured relations, and dependency mapping |

# 6. Logical Design - Relational Schema

## 6.1 Overview

The logical design translates the conceptual ER model into a fully relational schema consisting of 21 tables. Each table is defined with its attributes, primary key, foreign keys, and integrity constraints. The schema enforces referential integrity throughout, uses surrogate auto-increment keys as the default primary key strategy (with composite keys where semantically appropriate), and separates user roles (Buyer, Seller) into specialized subtypes linked to a central User table via shared primary key inheritance.

## 6.2 Relational Schema Notation

The schema below uses the following notation:

- **Bold + Underlined** attribute = Primary Key (PK)
- *Dashed underline* attribute = Foreign Key (FK)
- **Bold + dashed** = PK and FK combined (shared key inheritance or composite key)

## 6.3 Uniqueness/Not Null Changes

### Uniqueness Decision

**First**, attributes explicitly marked as unique in the ER diagram such as User.email, Category.name, and Seller.bank_account are assigned UNIQUE constraints as they represent natural candidate keys with no permissible duplicates.

**Second**, all 1:1 relationships are enforced by placing a UNIQUE constraint on the foreign key of the dependent table; this applies to Cart.buyer_id, Order.shipment_id, Payment.order_id, and Transaction.payment_id, ensuring that no two records on the dependent side can reference the same parent record.

**Third**, composite UNIQUE constraints are applied where the combination of two columns must be unique rather than each column individually specifically UNIQUE(buyer_id, product_id) in Review to prevent duplicate reviews, UNIQUE(product_id, warehouse_id) in Inventory to prevent duplicate stock records, UNIQUE(product_id, attr_name) in ProductAttribute to prevent repeated attributes for the same product, and UNIQUE(category_id, name) in Subcategory to prevent duplicate subcategory names within the same category. Together these rules ensure that the relational schema faithfully enforces all uniqueness semantics expressed in the original ER model.

### Not Null Decision

**First**, all primary key attributes are implicitly NOT NULL by definition, as a primary key must uniquely and completely identify every row in a table**.**

**Second**, foreign keys representing total participation relationships in the ER diagram are marked NOT NULL meaning the dependent entity cannot exist without its associated parent. For example, Address.user_id is NOT NULL because every address must belong to a user, Product.seller_id is NOT NULL because every product must have a seller, and Inventory.product_id and

Inventory.warehouse_id are both NOT NULL because an inventory record cannot exist without both a product and a warehouse.

**Third**, attributes critical to core business operations are enforced as NOT NULL regardless of whether they carry a relationship these include Product.price, Product.stock, and Product.status since a product without pricing or availability information is operationally incomplete, Order.status and Order.total_amount since an order record must always reflect a valid state and financial value, and Warehouse.name, Warehouse.location, and Warehouse.capacity since a warehouse without identifying or operational information serves no purpose in the system. Attributes that are genuinely optional in real-world scenarios such as User.phone, Address.country, Shipment.shipped_at, and Review.comment are intentionally left nullable, reflecting the principle that NOT NULL should only be enforced where the absence of a value would create an incomplete or invalid record.

## 6.4 Key Design Decisions

**Supertype/Subtype inheritance (User → Buyer, Seller):** Rather than a single flat User table, role-specific attributes are separated into Buyer and Seller subtypes. Both share user_id as a PK/FK, inheriting identity from User. This eliminates nulls for inapplicable columns (e.g., a Buyer has no shop_name) and keeps each table focused.

**Composite PKs for junction tables (CartItem, OrderItem):** CartItem and OrderItem use composite primary keys (cart_id + product_id) and (order_id + product_id) respectively. This directly enforces the constraint that a product appears at most once per cart or order, without needing an extra UNIQUE constraint.

**Surrogate PK for Inventory:** Although a composite PK of (product_id, warehouse_id) would be semantically valid, a surrogate key (inventory_id) is used for consistency and to simplify potential foreign key references from audit or movement tables. The UNIQUE(product_id, warehouse_id) constraint preserves the same integrity guarantee.

**1:1 relationships via UNIQUE FK:** Cart–Buyer, Order–Shipment, and Order–Payment are all 1:1 relationships enforced by placing a UNIQUE constraint on the FK column rather than merging the tables. This keeps concerns separated and allows Shipment and Payment to be created independently before being linked to an Order.

**unit_price snapshot in OrderItem:** The unit_price stored in OrderItem captures the price at the time of purchase. This is intentional — product prices may change over time, and the order history must remain accurate regardless of future price updates.

## 6.5 Relational Schema

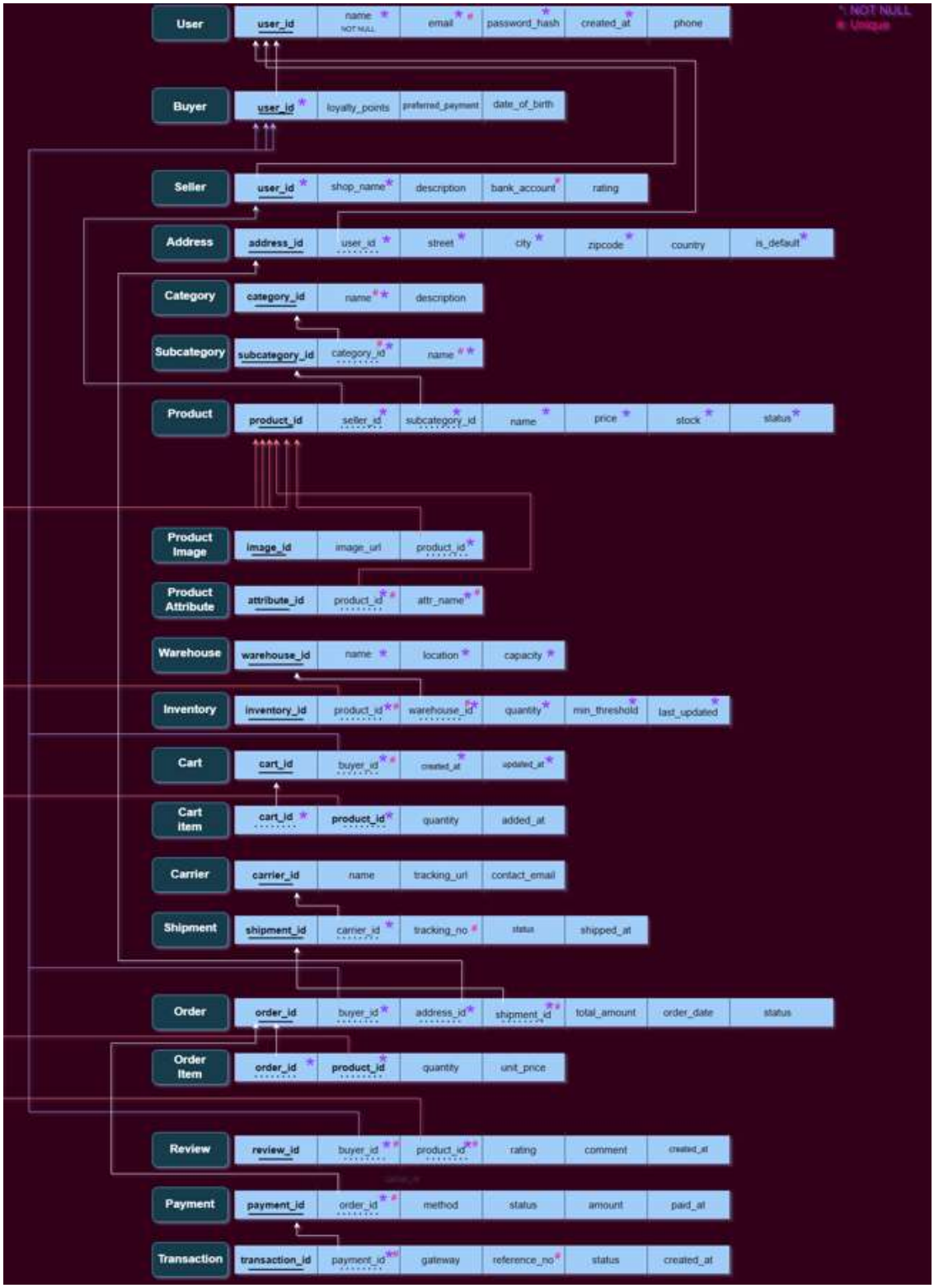



*Figure 1 Sub Part 1*


*Figure 2 Sub Part 2*


*Figure 3 Sub Part 3*

- The relational schema was derived from an ER diagram for an e-commerce system and fulfills all required constraints while ensuring normalization, consistency, and scalability.
- It includes 12+ entities (User, Buyer, Seller, Product, Cart, Order, Payment, Shipment, Inventory, Warehouse, Review, etc.), exceeding the required 6–8 entities and improving system modularity.
- Many-to-many relationships are resolved using associative entities such as OrderItem, CartItem, ProductAttribute, and Review, ensuring proper normalization and reduced redundancy.
- Specialization is implemented by dividing the User entity into Buyer and Seller, supporting role-based design.
- The schema uses a mix of simple, controlled (ENUM), and multi-valued attributes, with clearly defined 1:N and M:N relationships across entities.

For **data integrity and constraint handling**, primary keys and foreign keys are used throughout the schema. Additionally, a custom notation system is applied where:

- ⋆ Indicates **NOT NULL constraints**
- # indicates **UNIQUE constraints**

This ensures clarity in schema representation while maintaining strict data validation rules.

# 7. Normalization to 3NF

All 21 tables were analyzed and confirmed to satisfy First, Second, and Third Normal Form. The analysis below covers the most significant tables. The complete 3NF documentation for all tables is included in the attached 3NF_Form__.pdf.

## 7.1 Normal Form Definitions

- 1NF (First Normal Form): Every column holds a single atomic value; no repeating groups; each row is uniquely identifiable.
- 2NF (Second Normal Form): Table is in 1NF and every non-key attribute is fully dependent on the entire primary key (no partial dependencies - applies only to tables with composite PKs).
- 3NF (Third Normal Form): Table is in 2NF and no non-key attribute is transitively dependent on the primary key (no non-key column determines another non-key column).

## 7.2 Normalization Summary Table

| Table | 1NF | 2NF | 3NF | Key Justification |
|---|---|---|---|---|
| User | Done | Done | Done | Single PK. email/phone do not determine each other. |
| Buyer | Done | Done | Done | Single PK (user_id). All attrs depend solely on user_id. |
| Seller | Done | Done | Done | shop_name does not determine bank_account. |
| Address | Done | Done | Done | city does not determine country (stored independently). |
| Category | Done | Done | Done | name and description are independent of each other. |
| Subcategory | Done | Done | Done | category_id is FK, not a source of transitive dependency. |
| Product | Done | Done | Done | seller_id and subcategory_id are FKs. price ≠ f(name). |
| ProductImage | Done | Done | Done | Only 2 non-key attributes; neither determines the other. |
| ProductAttribute | Done | Done | Done | attr_name depends only on attribute_id. |
| Cart | Done | Done | Done | created_at/updated_at depend only on cart_id. |
| CartItem | Done | Done | Done | Composite PK. quantity depends on (cart_id, product_id) fully. |
| Inventory | Done | Done | Done | quantity and min_threshold are independent; both depend on PK. |
| Warehouse | Done | Done | Done | name does not determine location. |
| Carrier | Done | Done | Done | tracking_url and contact_email depend only on carrier_id. |
| Shipment | Done | Done | Done | tracking_num does not determine status. |
| Order | Done | Done | Done | All FKs. total_amount is derived (not a transitive dependency). |
| Buyer_Order | Done | Done | Done | Only FK columns; no transitive dependency possible. |
| OrderItem | Done | Done | Done | Composite PK. unit_price does not depend on quantity. |
| Payment | Done | Done | Done | method does not determine amount; status ≠ f(paid_at). |

| | | | | |
|---|---|---|---|---|
| Transaction | Done | Done | Done | gateway does not determine reference_no. |
| Review | Done | Done | Done | UNIQUE(buyer_id, product_id). rating ≠ f(comment). |

## 7.3 Detailed Analysis

### 7.3.1 CartItem (Composite PK)

CartItem has a composite PK of (cart_id, product_id). The only non-key attributes are quantity and added_at.

- 2NF Check: quantity cannot be determined by cart_id alone (we need to know which product) nor by product_id alone (we need to know which cart). Both non-key columns depend on the full composite key. In 2NF.
- 3NF Check: quantity does not determine added_at, and added_at does not determine quantity. No transitive dependency exists. In 3NF.

### 7.3.2 OrderItem (Composite PK)

OrderItem has composite PK (order_id, product_id) and non-key attributes quantity and unit_price.

- 2NF: quantity requires knowing both the specific order and the specific product (e.g., 3 units of product #5 in order #12). Neither column can be determined by either FK alone. In 2NF.
- 3NF: unit_price is a historical snapshot at purchase time; it does not functionally depend on quantity, and vice versa. In 3NF.

### 7.3.3 Inventory (Junction with Surrogate PK)

Inventory uses a surrogate PK (inventory_id) with a UNIQUE constraint on (product_id, warehouse_id). Non-key attributes are quantity, min_threshold, and last_updated.

- 2NF: Since a surrogate single-column PK is used, 2NF is trivially satisfied (no composite PK → no partial dependency possible).
- 3NF: quantity and min_threshold are independent values (one does not determine the other). Both depend only on the specific product-warehouse combination identified by inventory_id. In 3NF.

# 8. SQL Implementation - DDL (CREATE TABLE)

The following SQL scripts create the MarketHub database and all 21 tables with full constraints. The database uses Microsoft SQL Server T-SQL syntax.

## 8.1 Create Database

```
CREATE DATABASE MarketHubDB;
USE MarketHubDB;
```

## 8.2 Core Tables - User & Identity

```
CREATE TABLE [User] (
    user_id       INT IDENTITY(1,1) PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    password_hash VARCHAR(150) NOT NULL,
    phone         VARCHAR(20),
    created_at    DATETIME NOT NULL DEFAULT GETDATE()
);
```

```
CREATE TABLE Buyer (
    user_id          INT PRIMARY KEY,
    loyalty_points   INT NOT NULL DEFAULT 0,
    preferred_payment VARCHAR(50),
    date_of_birth    DATE NOT NULL,
    CONSTRAINT chk_buyer_age CHECK (DATEDIFF(YEAR, date_of_birth, GETDATE()) >= 16),
    CONSTRAINT fk_buyer_user FOREIGN KEY (user_id) REFERENCES [User](user_id) ON DELETE CASCADE
);
```

```
CREATE TABLE Seller (
    user_id      INT PRIMARY KEY,
    shop_name    VARCHAR(150) NOT NULL,
    description  TEXT,
    bank_account VARCHAR(50),
    rating       DECIMAL(3,2) CHECK (rating >= 0 AND rating <= 5),
    CONSTRAINT fk_seller_user FOREIGN KEY (user_id) REFERENCES [User](user_id) ON DELETE CASCADE
);
```

```
CREATE TABLE Address (
    address_id INT IDENTITY(1,1) PRIMARY KEY,
    user_id    INT NOT NULL,
    street     VARCHAR(200) NOT NULL,
    city       VARCHAR(100) NOT NULL,
    country    VARCHAR(100) NOT NULL,
    zip_code   VARCHAR(20),
    is_default BIT NOT NULL DEFAULT 0,
    CONSTRAINT fk_address_user FOREIGN KEY (user_id) REFERENCES [User](user_id) ON DELETE
CASCADE
);
```

## 8.3 Product Catalog Tables

```
CREATE TABLE Category (
   category_id INT IDENTITY(1,1) PRIMARY KEY,
   name        VARCHAR(100) NOT NULL UNIQUE,
   description TEXT
);
```

```
CREATE TABLE Subcategory (
   subcategory_id INT IDENTITY(1,1) PRIMARY KEY,
   category_id    INT NOT NULL,
   name           VARCHAR(100) NOT NULL,
   CONSTRAINT fk_subcategory_category
      FOREIGN KEY (category_id) REFERENCES Category(category_id) ON DELETE CASCADE
);
```

```
CREATE TABLE Product (
   product_id     INT IDENTITY(1,1) PRIMARY KEY,
   seller_id      INT NOT NULL,
   subcategory_id INT NOT NULL,
   name           VARCHAR(200) NOT NULL,
   price          DECIMAL(10,2) NOT NULL,
   stock          INT NOT NULL DEFAULT 0,
   status         VARCHAR(20) NOT NULL DEFAULT 'active',
   CONSTRAINT chk_product_price  CHECK (price > 0),
   CONSTRAINT chk_product_stock  CHECK (stock >= 0),
   CONSTRAINT chk_product_status CHECK (status IN ('active','inactive','deleted')),
   CONSTRAINT fk_product_seller      FOREIGN KEY (seller_id) REFERENCES Seller(user_id),
   CONSTRAINT fk_product_subcategory FOREIGN KEY (subcategory_id) REFERENCES
Subcategory(subcategory_id)
);
```

```
CREATE TABLE ProductImage (
   image_id   INT IDENTITY(1,1) PRIMARY KEY,
   product_id INT NOT NULL,
   image_url  VARCHAR(255) NOT NULL,
   CONSTRAINT fk_productimage_product FOREIGN KEY (product_id) REFERENCES Product(product_id) ON
DELETE CASCADE
);
```

```
CREATE TABLE ProductAttribute (
   attribute_id INT IDENTITY(1,1) PRIMARY KEY,
   product_id   INT NOT NULL,
   attr_name    VARCHAR(100) NOT NULL,
   CONSTRAINT fk_productattr_product FOREIGN KEY (product_id) REFERENCES Product(product_id) ON
DELETE CASCADE
);
```

## 8.4 Cart, Inventory & Logistics Tables

```
CREATE TABLE Cart (
   cart_id    INT IDENTITY(1,1) PRIMARY KEY,
   buyer_id   INT NOT NULL UNIQUE,
   created_at DATETIME NOT NULL DEFAULT GETDATE(),
   updated_at DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT fk_cart_buyer FOREIGN KEY (buyer_id) REFERENCES Buyer(user_id) ON DELETE
CASCADE
);
```

```
CREATE TABLE CartItem (
   cart_id    INT NOT NULL,
   product_id INT NOT NULL,
   quantity   INT NOT NULL DEFAULT 1,
   added_at   DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT pk_cartitem      PRIMARY KEY (cart_id, product_id),
   CONSTRAINT chk_cartitem_qty CHECK (quantity >= 1),
   CONSTRAINT fk_cartitem_cart    FOREIGN KEY (cart_id)    REFERENCES Cart(cart_id) ON DELETE
CASCADE,
   CONSTRAINT fk_cartitem_product FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

```
CREATE TABLE Warehouse (
   warehouse_id INT IDENTITY(1,1) PRIMARY KEY,
   name         VARCHAR(150) NOT NULL,
   location     VARCHAR(255),
   capacity     INT CHECK (capacity > 0)
);
```

```
CREATE TABLE Inventory (
   inventory_id  INT IDENTITY(1,1) PRIMARY KEY,
   product_id    INT NOT NULL,
   warehouse_id  INT NOT NULL,
   quantity      INT NOT NULL DEFAULT 0,
   min_threshold INT NOT NULL DEFAULT 5,
   last_updated  DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT uq_inventory         UNIQUE (product_id, warehouse_id),
   CONSTRAINT chk_inventory_qty    CHECK (quantity >= 0),
   CONSTRAINT chk_inventory_thresh CHECK (min_threshold >= 0),
   CONSTRAINT fk_inventory_product   FOREIGN KEY (product_id)   REFERENCES Product(product_id),
   CONSTRAINT fk_inventory_warehouse FOREIGN KEY (warehouse_id) REFERENCES
Warehouse(warehouse_id)
);
```

```
CREATE TABLE Carrier (
   carrier_id    INT IDENTITY(1,1) PRIMARY KEY,
   name          VARCHAR(100) NOT NULL,
   tracking_url  VARCHAR(255),
   contact_email VARCHAR(150)
);
```

```
CREATE TABLE Shipment (
   shipment_id  INT IDENTITY(1,1) PRIMARY KEY,
   carrier_id   INT NOT NULL,
   tracking_num VARCHAR(100),
   shipped_at   DATETIME,
   status       VARCHAR(20) NOT NULL DEFAULT 'pending',
   CONSTRAINT chk_shipment_status CHECK (status IN ('pending','dispatched','in_transit','delivered','returned')),
   CONSTRAINT fk_shipment_carrier  FOREIGN KEY (carrier_id) REFERENCES Carrier(carrier_id)
);
```

## 8.5 Order & Payment Tables

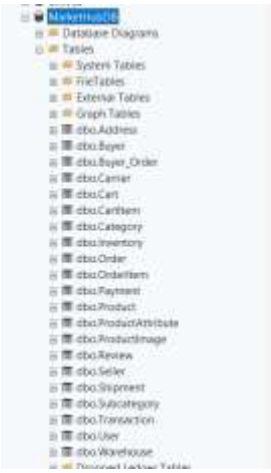
```
CREATE TABLE [Order] (
   order_id     INT IDENTITY(1,1) PRIMARY KEY,
   buyer_id     INT NOT NULL,
   address_id   INT NOT NULL,
   shipment_id  INT NOT NULL UNIQUE,
   order_date   DATETIME NOT NULL DEFAULT GETDATE(),
   total_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
   status       VARCHAR(20) NOT NULL DEFAULT 'pending',
   CONSTRAINT chk_order_status
      CHECK (status IN ('pending','confirmed','shipped','delivered','cancelled','returned')),
   CONSTRAINT fk_order_buyer    FOREIGN KEY (buyer_id)    REFERENCES Buyer(user_id),
   CONSTRAINT fk_order_address  FOREIGN KEY (address_id)  REFERENCES Address(address_id),
   CONSTRAINT fk_order_shipment FOREIGN KEY (shipment_id) REFERENCES Shipment(shipment_id)
);
```

```
CREATE TABLE OrderItem (
   order_id   INT NOT NULL,
   product_id INT NOT NULL,
   quantity   INT NOT NULL,
   unit_price DECIMAL(10,2) NOT NULL,
   CONSTRAINT pk_orderitem      PRIMARY KEY (order_id, product_id),
   CONSTRAINT chk_oi_qty        CHECK (quantity >= 1),
   CONSTRAINT chk_oi_price      CHECK (unit_price > 0),
   CONSTRAINT fk_oi_order       FOREIGN KEY (order_id)   REFERENCES [Order](order_id),
   CONSTRAINT fk_oi_product     FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```

```
CREATE TABLE Payment (
   payment_id INT IDENTITY(1,1) PRIMARY KEY,
   order_id   INT NOT NULL UNIQUE,
   method     VARCHAR(50) NOT NULL,
   status     VARCHAR(20) NOT NULL DEFAULT 'pending',
   amount     DECIMAL(12,2) NOT NULL,
   paid_at    DATETIME,
   CONSTRAINT chk_payment_status CHECK (status IN ('pending','completed','failed','refunded')),
   CONSTRAINT chk_payment_amount CHECK (amount > 0),
   CONSTRAINT fk_payment_order    FOREIGN KEY (order_id) REFERENCES [Order](order_id)
);
```

```
CREATE TABLE [Transaction] (
   transaction_id INT IDENTITY(1,1) PRIMARY KEY,
   payment_id     INT NOT NULL,
   gateway        VARCHAR(100) NOT NULL,
   reference_no   VARCHAR(100),
   status         VARCHAR(20) NOT NULL DEFAULT 'pending',
   created_at     DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT chk_tx_status  CHECK (status IN ('pending','success','failed')),
   CONSTRAINT fk_tx_payment  FOREIGN KEY (payment_id) REFERENCES Payment(payment_id) ON DELETE
CASCADE
);
```

```
CREATE TABLE Review (
   review_id  INT IDENTITY(1,1) PRIMARY KEY,
   buyer_id   INT NOT NULL,
   product_id INT NOT NULL,
   rating     TINYINT NOT NULL,
   comment    TEXT,
   created_at DATETIME NOT NULL DEFAULT GETDATE(),
   CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5),
   CONSTRAINT uq_review         UNIQUE (buyer_id, product_id),
   CONSTRAINT fk_review_buyer   FOREIGN KEY (buyer_id)   REFERENCES Buyer(user_id),
   CONSTRAINT fk_review_product FOREIGN KEY (product_id) REFERENCES Product(product_id)
);
```



# 9. SQL Implementation - DML (INSERT / UPDATE / DELETE)

## 9.1 Sample Data Insertion

```
-- Insert Users
INSERT INTO [User] (name, email, password_hash, phone) VALUES
  ('Ali Hassan',   'ali@marketplace.pk',    'hash_ali',   '03001234567'),
  ('Sara Malik',   'sara@marketplace.pk',   'hash_sara',  '03111234567'),
  ('Bilal Ahmed',  'bilal@marketplace.pk',  'hash_bilal', '03211234567'),
  ('Nadia Khan',   'nadia@marketplace.pk',  'hash_nadia', '03311234567'),
  ('Usman Raza',   'usman@marketplace.pk',  'hash_usman', '03411234567');
```

```
-- Insert Buyers
INSERT INTO Buyer (user_id, loyalty_points, preferred_payment, date_of_birth) VALUES
  (1, 0, 'credit_card', '2000-05-15'),
  (2, 150, 'easypaisa', '1998-11-20');
```

```
-- Insert Sellers
INSERT INTO Seller (user_id, shop_name, description, bank_account) VALUES
  (3, 'TechZone PK', 'Premium electronics retailer', 'HBL-003-PK'),
  (4, 'FashionHub', 'Trendy clothing and accessories', 'MCB-004-PK'),
  (5, 'HomeEssentials', 'Home goods and appliances', 'UBL-005-PK');
```

```
-- Insert Categories and Subcategories
INSERT INTO Category (name, description) VALUES
  ('Electronics', 'Phones, laptops, gadgets'),
  ('Clothing', 'Apparel and accessories'),
  ('Home & Living', 'Home decor and appliances');
```

```
INSERT INTO Subcategory (category_id, name) VALUES
  (1, 'Smartphones'), (1, 'Laptops'), (1, 'Accessories'),
  (2, 'Men Fashion'), (2, 'Women Fashion'),
  (3, 'Kitchen'), (3, 'Bedding');
```

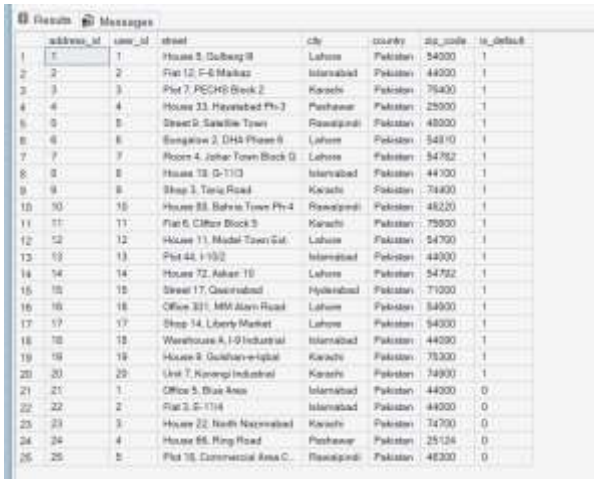
```
-- Insert Products
INSERT INTO Product (seller_id, subcategory_id, name, price, stock, status) VALUES
  (3, 1, 'Samsung Galaxy A55',   62999.00, 50, 'active'),
  (3, 2, 'Dell Inspiron 15',    109999.00, 20, 'active'),
  (3, 3, 'USB-C Hub 7-in-1',     4999.00, 100, 'active'),
  (4, 4, 'Men Kurta Shalwar',    3499.00,  80, 'active'),
  (4, 5, 'Women Lawn Suit',      2999.00,  60, 'active'),
  (5, 6, 'Non-Stick Cookware Set', 7500.00, 30, 'active'),
  (5, 7, 'King Size Comforter',  8999.00,  25, 'active');
```

```
-- Insert Carrier and Warehouse
INSERT INTO Carrier (name, tracking_url, contact_email) VALUES
  ('TCS Pakistan', 'https://tcs.com.pk/track', 'support@tcs.com.pk'),
```

```
('Leopards Courier', 'https://leopardscourier.com/track', 'cs@leopards.com.pk');

INSERT INTO Warehouse (name, location, capacity) VALUES
 ('Rawalpindi Central', 'Industrial Area, Rawalpindi', 10000),
 ('Lahore Hub', 'Sundar Industrial Estate, Lahore', 15000),
 ('Karachi South', 'SITE Area, Karachi', 20000);
```

Address:



Buyer:

Results Messages

| | user_id | loyalty_points | preferred_payment | date_of_birth |
|---|---|---|---|---|
| 1 | 1 | 120 | Credit Card | 1995-03-12 |
| 2 | 2 | 350 | EasyPaisa | 1998-07-22 |
| 3 | 3 | 80 | Cash on Del | 1990-11-05 |
| 4 | 4 | 500 | Credit Card | 1993-01-18 |
| 5 | 5 | 60 | JazzCash | 2000-06-30 |
| 6 | 6 | 200 | Debit Card | 1997-09-14 |
| 7 | 7 | 15 | Cash on Del | 1996-04-25 |
| 8 | 8 | 430 | Credit Card | 1994-12-02 |
| 9 | 9 | 90 | EasyPaisa | 1999-08-17 |
| 10 | 10 | 275 | JazzCash | 1992-05-08 |
| 11 | 11 | 45 | Debit Card | 2001-02-28 |
| 12 | 12 | 680 | Credit Card | 1988-10-19 |
| 13 | 13 | 110 | Cash on Del | 2002-03-07 |
| 14 | 14 | 320 | EasyPaisa | 1991-07-13 |
| 15 | 15 | 55 | Credit Card | 2003-11-21 |

Carrier:

Results Messages

| | carrier_id | name | tracking_url | contact_email |
|---|---|---|---|---|
| 1 | 1 | TCS Courier | https://track.tcs.com.pk/?id= | support@tcs.com.pk |
| 2 | 2 | Leopards | https://leopardscourier.com/track/? | cs@leopardscourier.com |
| 3 | 3 | M&P Express | https://mpak.com/track/?id= | info@mpak.com |
| 4 | 4 | BlueEx | https://blueex.com/tracking/?id= | help@blueex.com |
| 5 | 5 | Trax Logistics | https://app.traxlogistics.com/?id= | ops@traxlogistics.com |

Cart:

Results Messages

| | cart_id | buyer_id | created_at | updated_at |
|---|---|---|---|---|
| 1 | 1 | 1 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 2 | 2 | 2 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 3 | 3 | 3 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 4 | 4 | 4 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 5 | 5 | 5 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 6 | 6 | 6 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 7 | 7 | 7 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 8 | 8 | 8 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 9 | 9 | 9 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 10 | 10 | 10 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 11 | 11 | 11 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 12 | 12 | 12 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 13 | 13 | 13 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 14 | 14 | 14 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |
| 15 | 15 | 15 | 2026-04-19 21:37:34.970 | 2026-04-19 21:37:34.970 |

Cart Item:

Results Messages

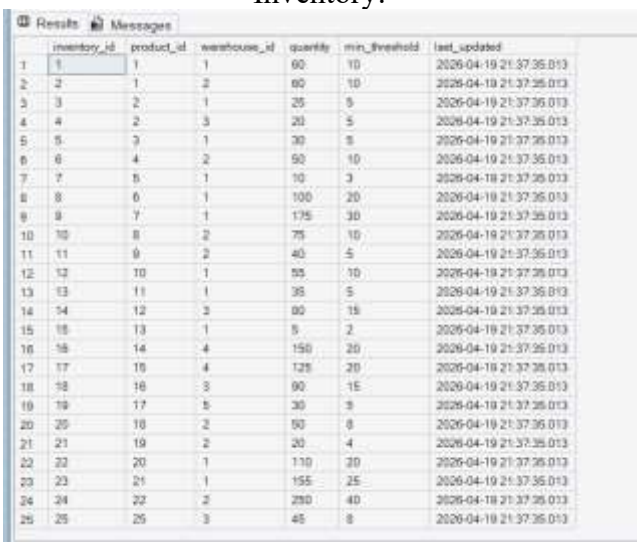
| | cart_id | product_id | quantity | added_at |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 2026-04-19 21:37:34.990 |
| 2 | 1 | 3 | 1 | 2026-04-19 21:37:34.990 |
| 3 | 2 | 7 | 2 | 2026-04-19 21:37:34.990 |
| 4 | 2 | 8 | 1 | 2026-04-19 21:37:34.990 |
| 5 | 3 | 14 | 1 | 2026-04-19 21:37:34.990 |
| 6 | 3 | 15 | 2 | 2026-04-19 21:37:34.990 |
| 7 | 4 | 10 | 1 | 2026-04-19 21:37:34.990 |
| 8 | 4 | 12 | 3 | 2026-04-19 21:37:34.990 |
| 9 | 5 | 18 | 1 | 2026-04-19 21:37:34.990 |
| 10 | 5 | 19 | 1 | 2026-04-19 21:37:34.990 |
| 11 | 6 | 20 | 2 | 2026-04-19 21:37:34.990 |
| 12 | 6 | 21 | 1 | 2026-04-19 21:37:34.990 |
| 13 | 7 | 6 | 1 | 2026-04-19 21:37:34.990 |
| 14 | 7 | 9 | 1 | 2026-04-19 21:37:34.990 |
| 15 | 8 | 2 | 1 | 2026-04-19 21:37:34.990 |
| 16 | 8 | 4 | 1 | 2026-04-19 21:37:34.990 |
| 17 | 9 | 22 | 4 | 2026-04-19 21:37:34.990 |
| 18 | 9 | 23 | 1 | 2026-04-19 21:37:34.990 |
| 19 | 10 | 11 | 1 | 2026-04-19 21:37:34.990 |
| 20 | 10 | 17 | 1 | 2026-04-19 21:37:34.990 |
| 21 | 11 | 24 | 1 | 2026-04-19 21:37:34.990 |
| 22 | 12 | 5 | 1 | 2026-04-19 21:37:34.990 |
| 23 | 13 | 16 | 2 | 2026-04-19 21:37:34.990 |
| 24 | 14 | 13 | 1 | 2026-04-19 21:37:34.990 |
| 25 | 15 | 25 | 2 | 2026-04-19 21:37:34.990 |

Category:

Results Messages

| | category_id | name | description |
|---|---|---|---|
| 1 | 1 | Electronics | Gadgets, devices, and accessories |
| 2 | 2 | Fashion | Clothing, footwear, and accessories |
| 3 | 3 | Home & Living | Furniture, kitchenware, and décor |
| 4 | 4 | Books | Fiction, non-fiction, and educational |
| 5 | 5 | Sports | Equipment, apparel, and accessories |
| 6 | 6 | Beauty | Skincare, makeup, and wellness |
| 7 | 7 | Groceries | Food, beverages, and daily essentials |
| 8 | 8 | Toys & Games | Educational and recreational products |

Inventory:



Results Messages

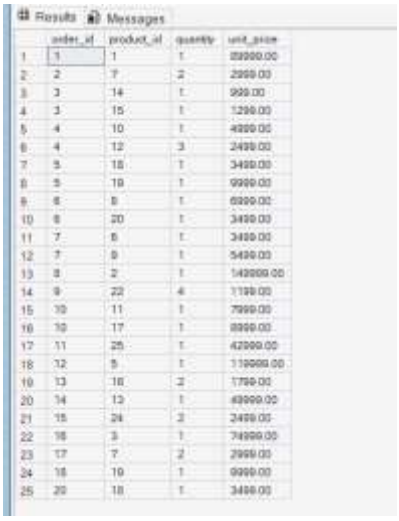
| | inventory_id | product_id | warehouse_id | quantity | min_threshold | last_updated |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 60 | 10 | 2026-04-19 21:37:35.013 |
| 2 | 2 | 1 | 2 | 60 | 10 | 2026-04-19 21:37:35.013 |
| 3 | 3 | 2 | 1 | 25 | 5 | 2026-04-19 21:37:35.013 |
| 4 | 4 | 2 | 3 | 20 | 5 | 2026-04-19 21:37:35.013 |
| 5 | 5 | 3 | 1 | 30 | 5 | 2026-04-19 21:37:35.013 |
| 6 | 6 | 4 | 2 | 50 | 10 | 2026-04-19 21:37:35.013 |
| 7 | 7 | 5 | 1 | 10 | 3 | 2026-04-19 21:37:35.013 |
| 8 | 8 | 6 | 1 | 100 | 20 | 2026-04-19 21:37:35.013 |
| 9 | 9 | 7 | 1 | 175 | 30 | 2026-04-19 21:37:35.013 |
| 10 | 10 | 8 | 2 | 75 | 10 | 2026-04-19 21:37:35.013 |
| 11 | 11 | 9 | 2 | 40 | 5 | 2026-04-19 21:37:35.013 |
| 12 | 12 | 10 | 1 | 55 | 10 | 2026-04-19 21:37:35.013 |
| 13 | 13 | 11 | 1 | 35 | 5 | 2026-04-19 21:37:35.013 |
| 14 | 14 | 12 | 3 | 80 | 15 | 2026-04-19 21:37:35.013 |
| 15 | 15 | 13 | 1 | 5 | 2 | 2026-04-19 21:37:35.013 |
| 16 | 16 | 14 | 4 | 150 | 20 | 2026-04-19 21:37:35.013 |
| 17 | 17 | 15 | 4 | 125 | 20 | 2026-04-19 21:37:35.013 |
| 18 | 18 | 16 | 3 | 90 | 15 | 2026-04-19 21:37:35.013 |
| 19 | 19 | 17 | 5 | 30 | 5 | 2026-04-19 21:37:35.013 |
| 20 | 20 | 18 | 2 | 50 | 8 | 2026-04-19 21:37:35.013 |
| 21 | 21 | 19 | 2 | 20 | 4 | 2026-04-19 21:37:35.013 |
| 22 | 22 | 20 | 1 | 110 | 20 | 2026-04-19 21:37:35.013 |
| 23 | 23 | 21 | 1 | 155 | 25 | 2026-04-19 21:37:35.013 |
| 24 | 24 | 22 | 2 | 250 | 40 | 2026-04-19 21:37:35.013 |
| 25 | 25 | 25 | 3 | 45 | 8 | 2026-04-19 21:37:35.013 |

Order:

Results Messages

| | order_id | buyer_id | address_id | shipment_id | order_date | total_amount | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 2026-04-19 21:37:35.063 | 89999.00 | delivered |
| 2 | 2 | 2 | 2 | 2 | 2026-04-19 21:37:35.063 | 5998.00 | delivered |
| 3 | 3 | 3 | 3 | 3 | 2026-04-19 21:37:35.063 | 2298.00 | delivered |
| 4 | 4 | 4 | 4 | 4 | 2026-04-19 21:37:35.063 | 12498.00 | delivered |
| 5 | 5 | 5 | 5 | 5 | 2026-04-19 21:37:35.063 | 13498.00 | delivered |
| 6 | 6 | 6 | 6 | 6 | 2026-04-19 21:37:35.063 | 5498.00 | delivered |
| 7 | 7 | 7 | 7 | 7 | 2026-04-19 21:37:35.063 | 11498.00 | confirmed |
| 8 | 8 | 8 | 8 | 8 | 2026-04-19 21:37:35.063 | 149999.00 | shipped |
| 9 | 9 | 9 | 9 | 9 | 2026-04-19 21:37:35.063 | 4796.00 | pending |
| 10 | 10 | 10 | 10 | 10 | 2026-04-19 21:37:35.063 | 12498.00 | pending |
| 11 | 11 | 11 | 11 | 11 | 2026-04-19 21:37:35.063 | 42999.00 | delivered |
| 12 | 12 | 12 | 12 | 12 | 2026-04-19 21:37:35.063 | 119999.00 | delivered |
| 13 | 13 | 13 | 13 | 13 | 2026-04-19 21:37:35.063 | 3598.00 | shipped |
| 14 | 14 | 14 | 14 | 14 | 2026-04-19 21:37:35.063 | 49999.00 | pending |
| 15 | 15 | 15 | 15 | 15 | 2026-04-19 21:37:35.063 | 4998.00 | delivered |
| 16 | 16 | 1 | 21 | 16 | 2026-04-19 21:37:35.063 | 74999.00 | delivered |
| 17 | 17 | 2 | 22 | 17 | 2026-04-19 21:37:35.063 | 7998.00 | returned |
| 18 | 18 | 3 | 23 | 18 | 2026-04-19 21:37:35.063 | 9999.00 | pending |
| 19 | 19 | 4 | 24 | 19 | 2026-04-19 21:37:35.063 | 19999.00 | shipped |
| 20 | 20 | 5 | 25 | 20 | 2026-04-19 21:37:35.063 | 3499.00 | confirmed |

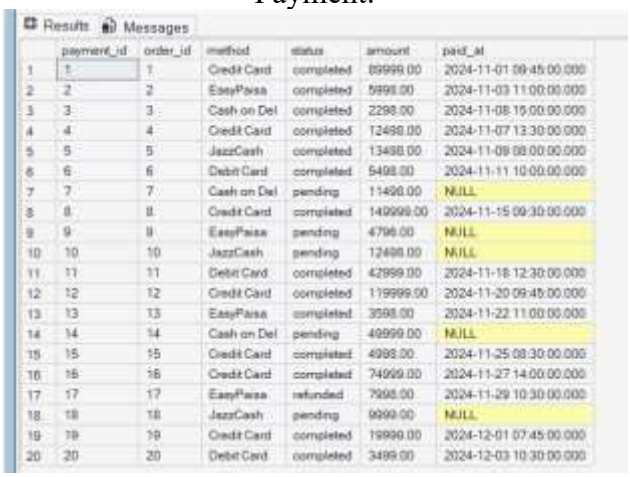
OrderItem:



Results Messages

| | order_id | product_id | quantity | unit_price |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 89999.00 |
| 2 | 2 | 7 | 2 | 2999.00 |
| 3 | 3 | 14 | 1 | 999.00 |
| 4 | 3 | 15 | 1 | 1299.00 |
| 5 | 4 | 10 | 1 | 4999.00 |
| 6 | 4 | 12 | 3 | 2499.00 |
| 7 | 5 | 18 | 1 | 3499.00 |
| 8 | 5 | 19 | 1 | 9999.00 |
| 9 | 6 | 8 | 1 | 5999.00 |
| 10 | 6 | 20 | 1 | 3499.00 |
| 11 | 7 | 6 | 1 | 3499.00 |
| 12 | 7 | 9 | 1 | 5499.00 |
| 13 | 8 | 2 | 1 | 149999.00 |
| 14 | 9 | 22 | 4 | 1199.00 |
| 15 | 10 | 11 | 1 | 7999.00 |
| 16 | 10 | 17 | 1 | 9999.00 |
| 17 | 11 | 25 | 1 | 42999.00 |
| 18 | 12 | 5 | 1 | 119999.00 |
| 19 | 13 | 16 | 2 | 1799.00 |
| 20 | 14 | 13 | 1 | 49999.00 |
| 21 | 15 | 24 | 3 | 2499.00 |
| 22 | 16 | 3 | 1 | 74999.00 |
| 23 | 17 | 7 | 2 | 2999.00 |
| 24 | 18 | 19 | 1 | 9999.00 |
| 25 | 20 | 18 | 1 | 3499.00 |

Payment:



Results Messages

| | payment_id | order_id | method | status | amount | paid_at |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | Credit Card | completed | 89999.00 | 2024-11-01 09:45:00.000 |
| 2 | 2 | 2 | EasyPaisa | completed | 5999.00 | 2024-11-03 11:00:00.000 |
| 3 | 3 | 3 | Cash on Del | completed | 2298.00 | 2024-11-08 15:00:00.000 |
| 4 | 4 | 4 | Credit Card | completed | 12498.00 | 2024-11-07 13:30:00.000 |
| 5 | 5 | 5 | JazzCash | completed | 13498.00 | 2024-11-09 08:00:00.000 |
| 6 | 6 | 6 | Debit Card | completed | 5498.00 | 2024-11-11 10:00:00.000 |
| 7 | 7 | 7 | Cash on Del | pending | 11498.00 | NULL |
| 8 | 8 | 8 | Credit Card | completed | 149999.00 | 2024-11-15 09:30:00.000 |
| 9 | 9 | 9 | EasyPaisa | pending | 4796.00 | NULL |
| 10 | 10 | 10 | JazzCash | pending | 12498.00 | NULL |
| 11 | 11 | 11 | Debit Card | completed | 42999.00 | 2024-11-18 12:30:00.000 |
| 12 | 12 | 12 | Credit Card | completed | 119999.00 | 2024-11-20 09:45:00.000 |
| 13 | 13 | 13 | EasyPaisa | completed | 3598.00 | 2024-11-22 11:00:00.000 |
| 14 | 14 | 14 | Cash on Del | pending | 49999.00 | NULL |
| 15 | 15 | 15 | Credit Card | completed | 4998.00 | 2024-11-25 08:30:00.000 |
| 16 | 16 | 16 | Credit Card | completed | 74999.00 | 2024-11-27 14:00:00.000 |
| 17 | 17 | 17 | EasyPaisa | refunded | 7998.00 | 2024-11-29 10:30:00.000 |
| 18 | 18 | 18 | JazzCash | pending | 9299.00 | NULL |
| 19 | 19 | 19 | Credit Card | completed | 19999.00 | 2024-12-01 07:45:00.000 |
| 20 | 20 | 20 | Debit Card | completed | 3499.00 | 2024-12-03 10:30:00.000 |

Product:

Results Messages

| | product_id | seller_id | subcategory_id | name | price | stock | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 16 | 1 | Samsung Galaxy A55 5G | 89999.00 | 120 | active |
| 2 | 2 | 16 | 2 | HP Pavilion 15 Laptop i5 | 149999.00 | 45 | active |
| 3 | 3 | 16 | 3 | Sony WH-1000XM5 Headphones | 74999.00 | 60 | active |
| 4 | 4 | 16 | 4 | Xiaomi Smart Watch Pro | 19999.00 | 90 | active |
| 5 | 5 | 16 | 5 | Canon EOS 1500D DSLR | 119999.00 | 20 | active |
| 6 | 6 | 17 | 6 | Men Linen Kurta - Navy Blue | 3499.00 | 200 | active |
| 7 | 7 | 17 | 7 | Women Lawn Suit - Floral Print | 2999.00 | 350 | active |
| 8 | 8 | 17 | 8 | Running Shoes - White (Men) | 6999.00 | 150 | active |
| 9 | 9 | 17 | 9 | Leather Messenger Bag | 5499.00 | 80 | active |
| 10 | 10 | 18 | 10 | Anex Juicer & Blender AG-179 | 4999.00 | 110 | active |
| 11 | 11 | 18 | 11 | Soft Microfiber Comforter King Size | 7999.00 | 70 | active |
| 12 | 12 | 18 | 12 | Ceramic Vase Set - 3 Pieces | 2499.00 | 180 | active |
| 13 | 13 | 18 | 13 | Wooden 6-Seater Dining Table | 49999.00 | 10 | active |
| 14 | 14 | 19 | 14 | The Kite Runner - Khaled Hosseini | 999.00 | 300 | active |
| 15 | 15 | 19 | 15 | Atomic Habits - James Clear | 1299.00 | 250 | active |
| 16 | 16 | 19 | 16 | OA Level Mathematics Guide | 1799.00 | 180 | active |
| 17 | 17 | 20 | 17 | Gray Nicolls Cricket Bat GN600 | 8999.00 | 55 | active |
| 18 | 18 | 20 | 18 | Nike Strike Football Size 5 | 3499.00 | 100 | active |
| 19 | 19 | 20 | 19 | Adjustable Dumbbell Set 20kg | 9999.00 | 40 | active |
| 20 | 20 | 21 | 20 | Neutrogena Hydro Boost Moistur... | 3299.00 | 220 | active |
| 21 | 21 | 21 | 21 | Maybelline Fit Me Foundation | 2199.00 | 310 | active |
| 22 | 22 | 22 | 22 | Knorr Chicken Noodles Pack of 12 | 1199.00 | 500 | active |
| 23 | 23 | 23 | 24 | Ludo & Snake Ladder Family Edit... | 799.00 | 130 | active |
| 24 | 24 | 23 | 25 | Building Blocks Set 200pcs | 2499.00 | 95 | active |
| 25 | 25 | 24 | 1 | Realme C67 - 128GB | 42999.00 | 175 | active |

Product Attribute:

Results Messages

| | attribute_id | product_id | attr_name |
|---|---|---|---|
| 1 | 1 | 1 | Color: Awesome Iceblue |
| 2 | 2 | 1 | RAM: 8GB |
| 3 | 3 | 2 | Storage: 512GB SSD |
| 4 | 4 | 2 | Display: 15.6 inch FHD |
| 5 | 5 | 3 | Noise Cancellation: Yes |
| 6 | 6 | 4 | Battery Life: 14 days |
| 7 | 7 | 5 | Sensor: 24.1 MP APS-C |
| 8 | 8 | 6 | Material: 100% Linen |
| 9 | 9 | 7 | Fabric: Printed Lawn |
| 10 | 10 | 8 | Sole: Rubber Outsole |
| 11 | 11 | 9 | Material: Genuine Leather |
| 12 | 12 | 10 | Power: 400W |
| 13 | 13 | 11 | Fill: Microfiber Hollow |
| 14 | 14 | 12 | Finish: Glossy White |
| 15 | 15 | 13 | Wood: Sheesham Solid |
| 16 | 16 | 14 | Language: English |
| 17 | 17 | 15 | Format: Paperback |
| 18 | 18 | 16 | Edition: 2024 |
| 19 | 19 | 17 | Weight: 1.2 kg |
| 20 | 20 | 18 | Size: 5 |
| 21 | 21 | 19 | Material: Cast Iron |
| 22 | 22 | 20 | Skin Type: All Skin Types |
| 23 | 23 | 21 | Finish: Natural Matte |
| 24 | 24 | 22 | Weight: 80g each |
| 25 | 25 | 23 | Players: 2-4 |

Product Image:

Results Messages

| | image_id | product_id | image_url |
|---|---|---|---|
| 1 | 1 | 1 | https://cdn.markethub.pk/products/samsung-a55-fro... |
| 2 | 2 | 1 | https://cdn.markethub.pk/products/samsung-a55-ba... |
| 3 | 3 | 2 | https://cdn.markethub.pk/products/hp-pavilion-15.jpg |
| 4 | 4 | 3 | https://cdn.markethub.pk/products/sony-xm5.jpg |
| 5 | 5 | 4 | https://cdn.markethub.pk/products/xiaomi-watch-pro... |
| 6 | 6 | 5 | https://cdn.markethub.pk/products/canon-1500d.jpg |
| 7 | 7 | 6 | https://cdn.markethub.pk/products/linen-kurta-navy.jpg |
| 8 | 8 | 7 | https://cdn.markethub.pk/products/lawn-suit-floral.jpg |
| 9 | 9 | 8 | https://cdn.markethub.pk/products/running-shoes-w... |
| 10 | 10 | 9 | https://cdn.markethub.pk/products/leather-bag.jpg |
| 11 | 11 | 10 | https://cdn.markethub.pk/products/anex-juicer.jpg |
| 12 | 12 | 11 | https://cdn.markethub.pk/products/comforter-king.jpg |
| 13 | 13 | 12 | https://cdn.markethub.pk/products/ceramic-vase-set... |
| 14 | 14 | 13 | https://cdn.markethub.pk/products/dining-table-woo... |
| 15 | 15 | 14 | https://cdn.markethub.pk/products/kite-runner.jpg |
| 16 | 16 | 15 | https://cdn.markethub.pk/products/atomic-habits.jpg |
| 17 | 17 | 16 | https://cdn.markethub.pk/products/maths-guide.jpg |
| 18 | 18 | 17 | https://cdn.markethub.pk/products/gn500-bat.jpg |
| 19 | 19 | 18 | https://cdn.markethub.pk/products/nike-football.jpg |
| 20 | 20 | 19 | https://cdn.markethub.pk/products/dumbbell-set.jpg |
| 21 | 21 | 20 | https://cdn.markethub.pk/products/neutrogena-boost... |
| 22 | 22 | 21 | https://cdn.markethub.pk/products/maybelline-fitme.j... |
| 23 | 23 | 22 | https://cdn.markethub.pk/products/knorr-noodles.jpg |
| 24 | 24 | 23 | https://cdn.markethub.pk/products/ludo-snakes.jpg |
| 25 | 25 | 24 | https://cdn.markethub.pk/products/building-blocks.jpg |

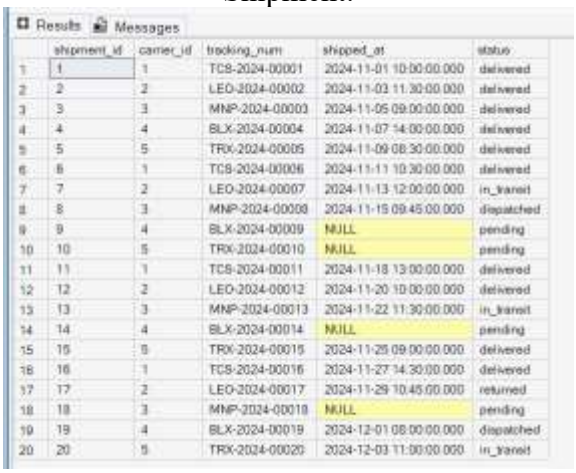
Review:

Seller:

Results Messages

| | user_id | shop_name | description | bank_account | rating |
|---|---|---|---|---|---|
| 1 | 16 | TechZone PK | Electronics & gadgets at best prices | PK36HABB0000001234567890 | 4.50 |
| 2 | 17 | FashionHub | Trendy clothing for men and women | PK36MEZN0000002345678901 | 4.20 |
| 3 | 18 | HomeNest | Quality home & kitchen products | PK36MUCB0000003456789012 | 3.90 |
| 4 | 19 | BookWorld | New and used books, all genres | PK36BAHL0000004567890123 | 4.70 |
| 5 | 20 | SportsKing | Sports gear and fitness equipment | PK36HABB0000005678901234 | 4.10 |
| 6 | 21 | BeautyBox | Skincare, makeup and wellness products | PK36MEZN0000006789012345 | 4.60 |
| 7 | 22 | GroceryDirect | Fresh and packaged grocery items | PK36MUCB0000007890123456 | 3.80 |
| 8 | 23 | ToyLand | Toys and games for all ages | PK36BAHL0000008901234567 | 4.30 |
| 9 | 24 | AutoParts Plus | Genuine and aftermarket auto parts | PK36HABB0000009012345678 | 3.70 |
| 10 | 25 | PetCorner | Pet food, accessories, and care products | PK36MEZN0000000123456789 | 4.40 |

Shipment:



Results Messages

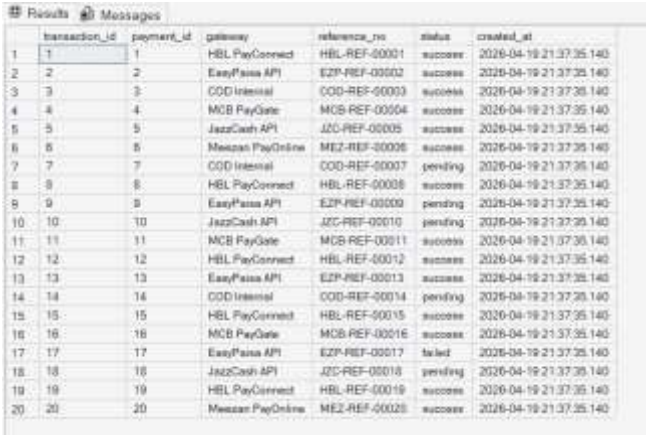
| | shipment_id | carrier_id | tracking_num | shipped_at | status |
|---|---|---|---|---|---|
| 1 | 1 | 1 | TCS-2024-00001 | 2024-11-01 10:00:00.000 | delivered |
| 2 | 2 | 2 | LEO-2024-00002 | 2024-11-03 11:30:00.000 | delivered |
| 3 | 3 | 3 | MNP-2024-00003 | 2024-11-05 09:00:00.000 | delivered |
| 4 | 4 | 4 | BLX-2024-00004 | 2024-11-07 14:00:00.000 | delivered |
| 5 | 5 | 5 | TRX-2024-00005 | 2024-11-09 08:30:00.000 | delivered |
| 6 | 6 | 1 | TCS-2024-00006 | 2024-11-11 10:30:00.000 | delivered |
| 7 | 7 | 2 | LEO-2024-00007 | 2024-11-13 12:00:00.000 | in_transit |
| 8 | 8 | 3 | MNP-2024-00008 | 2024-11-15 09:45:00.000 | dispatched |
| 9 | 9 | 4 | BLX-2024-00009 | NULL | pending |
| 10 | 10 | 5 | TRX-2024-00010 | NULL | pending |
| 11 | 11 | 1 | TCS-2024-00011 | 2024-11-18 13:00:00.000 | delivered |
| 12 | 12 | 2 | LEO-2024-00012 | 2024-11-20 10:00:00.000 | delivered |
| 13 | 13 | 3 | MNP-2024-00013 | 2024-11-22 11:30:00.000 | in_transit |
| 14 | 14 | 4 | BLX-2024-00014 | NULL | pending |
| 15 | 15 | 5 | TRX-2024-00015 | 2024-11-25 09:00:00.000 | delivered |
| 16 | 16 | 1 | TCS-2024-00016 | 2024-11-27 14:30:00.000 | delivered |
| 17 | 17 | 2 | LEO-2024-00017 | 2024-11-29 10:45:00.000 | returned |
| 18 | 18 | 3 | MNP-2024-00018 | NULL | pending |
| 19 | 19 | 4 | BLX-2024-00019 | 2024-12-01 08:00:00.000 | dispatched |
| 20 | 20 | 5 | TRX-2024-00020 | 2024-12-03 11:00:00.000 | in_transit |

Subcategory:

Results Messages

| | subcategory_id | category_id | name |
|---|---|---|---|
| 1 | 1 | 1 | Mobile Phones |
| 2 | 2 | 1 | Laptops |
| 3 | 3 | 1 | Headphones |
| 4 | 4 | 1 | Smartwatches |
| 5 | 5 | 1 | Cameras |
| 6 | 6 | 2 | Men Clothing |
| 7 | 7 | 2 | Women Clothing |
| 8 | 8 | 2 | Footwear |
| 9 | 9 | 2 | Bags & Wallets |
| 10 | 10 | 3 | Kitchen Appliances |
| 11 | 11 | 3 | Bedding |
| 12 | 12 | 3 | Home Décor |
| 13 | 13 | 3 | Furniture |
| 14 | 14 | 4 | Fiction |
| 15 | 15 | 4 | Self-Help |
| 16 | 16 | 4 | Academic |
| 17 | 17 | 5 | Cricket |
| 18 | 18 | 5 | Football |
| 19 | 19 | 5 | Fitness Equipment |
| 20 | 20 | 6 | Skincare |
| 21 | 21 | 6 | Makeup |
| 22 | 22 | 7 | Packaged Food |
| 23 | 23 | 7 | Fresh Produce |
| 24 | 24 | 8 | Board Games |
| 25 | 25 | 8 | Educational Toys |

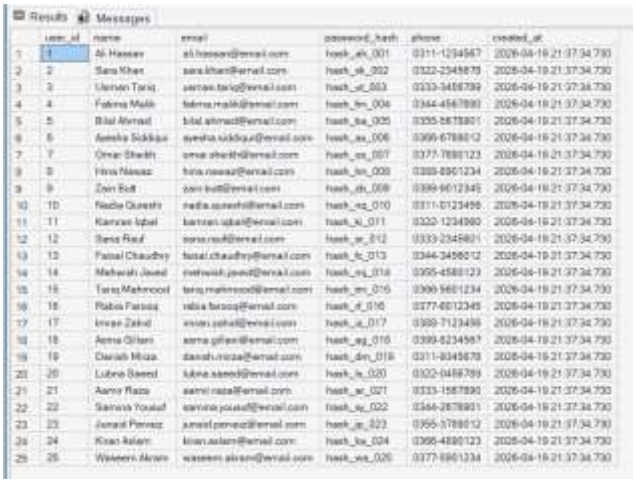
Transaction:



Results Messages

| | transaction_id | payment_id | gateway | reference_no | status | created_at |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | HBL PayConnect | HBL-REF-00001 | success | 2026-04-19 21:37:35.140 |
| 2 | 2 | 2 | EasyPaisa API | EZP-REF-00002 | success | 2026-04-19 21:37:35.140 |
| 3 | 3 | 3 | COD Internal | COD-REF-00003 | success | 2026-04-19 21:37:35.140 |
| 4 | 4 | 4 | MCB PayGate | MCB-REF-00004 | success | 2026-04-19 21:37:35.140 |
| 5 | 5 | 5 | JazzCash API | JZC-REF-00005 | success | 2026-04-19 21:37:35.140 |
| 6 | 6 | 6 | Meezan PayOnline | MEZ-REF-00006 | success | 2026-04-19 21:37:35.140 |
| 7 | 7 | 7 | COD Internal | COD-REF-00007 | pending | 2026-04-19 21:37:35.140 |
| 8 | 8 | 8 | HBL PayConnect | HBL-REF-00008 | success | 2026-04-19 21:37:35.140 |
| 9 | 9 | 9 | EasyPaisa API | EZP-REF-00009 | pending | 2026-04-19 21:37:35.140 |
| 10 | 10 | 10 | JazzCash API | JZC-REF-00010 | pending | 2026-04-19 21:37:35.140 |
| 11 | 11 | 11 | MCB PayGate | MCB-REF-00011 | success | 2026-04-19 21:37:35.140 |
| 12 | 12 | 12 | HBL PayConnect | HBL-REF-00012 | success | 2026-04-19 21:37:35.140 |
| 13 | 13 | 13 | EasyPaisa API | EZP-REF-00013 | success | 2026-04-19 21:37:35.140 |
| 14 | 14 | 14 | COD Internal | COD-REF-00014 | pending | 2026-04-19 21:37:35.140 |
| 15 | 15 | 15 | HBL PayConnect | HBL-REF-00015 | success | 2026-04-19 21:37:35.140 |
| 16 | 16 | 16 | MCB PayGate | MCB-REF-00016 | success | 2026-04-19 21:37:35.140 |
| 17 | 17 | 17 | EasyPaisa API | EZP-REF-00017 | failed | 2026-04-19 21:37:35.140 |
| 18 | 18 | 18 | JazzCash API | JZC-REF-00018 | pending | 2026-04-19 21:37:35.140 |
| 19 | 19 | 19 | HBL PayConnect | HBL-REF-00019 | success | 2026-04-19 21:37:35.140 |
| 20 | 20 | 20 | Meezan PayOnline | MEZ-REF-00020 | success | 2026-04-19 21:37:35.140 |

User:



Warehouse:

Results | Messages

| | warehouse_id | name | location | capacity |
|---|---|---|---|---|
| 1 | 1 | Lahore Central WH | Quaid-e-Azam Industrial Estate, Lahore | 5000 |
| 2 | 2 | Karachi South WH | Korangi Industrial Area, Karachi | 8000 |
| 3 | 3 | Islamabad North WH | I-9 Industrial Zone, Islamabad | 3000 |
| 4 | 4 | Rawalpindi WH | Chakra Road Industrial, Rawalpindi | 2000 |
| 5 | 5 | Peshawar WH | Hayatabad Industrial Estate, Peshawar | 1500 |

## 9.2 UPDATE Statements

```
-- Update product price after sale
UPDATE Product SET price = 59999.00 WHERE product_id = 1;
```

```
-- Update order status after dispatch
UPDATE [Order] SET status = 'shipped' WHERE order_id = 1;
```

```
-- Update buyer loyalty points after purchase
UPDATE Buyer SET loyalty_points = loyalty_points + 100 WHERE user_id = 1;
```

```
-- Soft-delete a product
UPDATE Product SET status = 'deleted' WHERE product_id = 7;
```

Product:

Results Messages

| | product_id | seller_id | subcategory_id | name | price | stock | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 16 | 1 | Samsung Galaxy A55 5G | 95000.00 | 100 | active |

Order:

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 9 | 9 | 9 | 9 | 9 | 2026-04-19 21:37:35.063 | 4796.00 | shipped |
| 10 | 10 | 10 | 10 | 10 | 2026-04-19 21:37:35.063 | 12498.00 | pending |

Shipment:

| | | | | | |
|---|---|---|---|---|---|
| 10 | 10 | 5 | TRX-2024-00010 | 2026-04-20 22:12:57.717 | in_transit |

User:

Results Messages

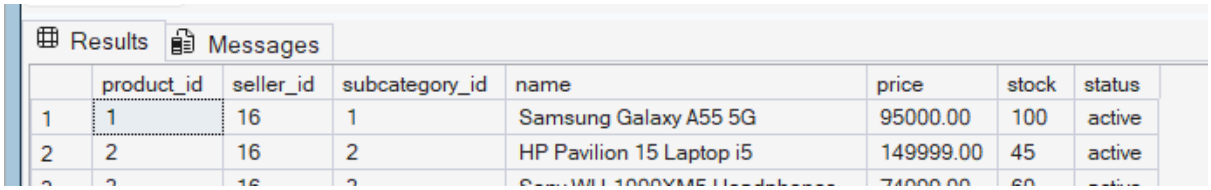
| | user_id | name | email | password_hash | phone | created_at |
|---|---|---|---|---|---|---|
| 1 | 1 | Ali Hassan Updated | ali.hassan@email.com | hash_ah_001 | 0300-1111111 | 2026-04-19 21:37:34.730 |

## 9.3 DELETE Statements

```
-- Delete a cart item (buyer removed item from cart)
DELETE FROM CartItem WHERE cart_id = 1 AND product_id = 3;
```

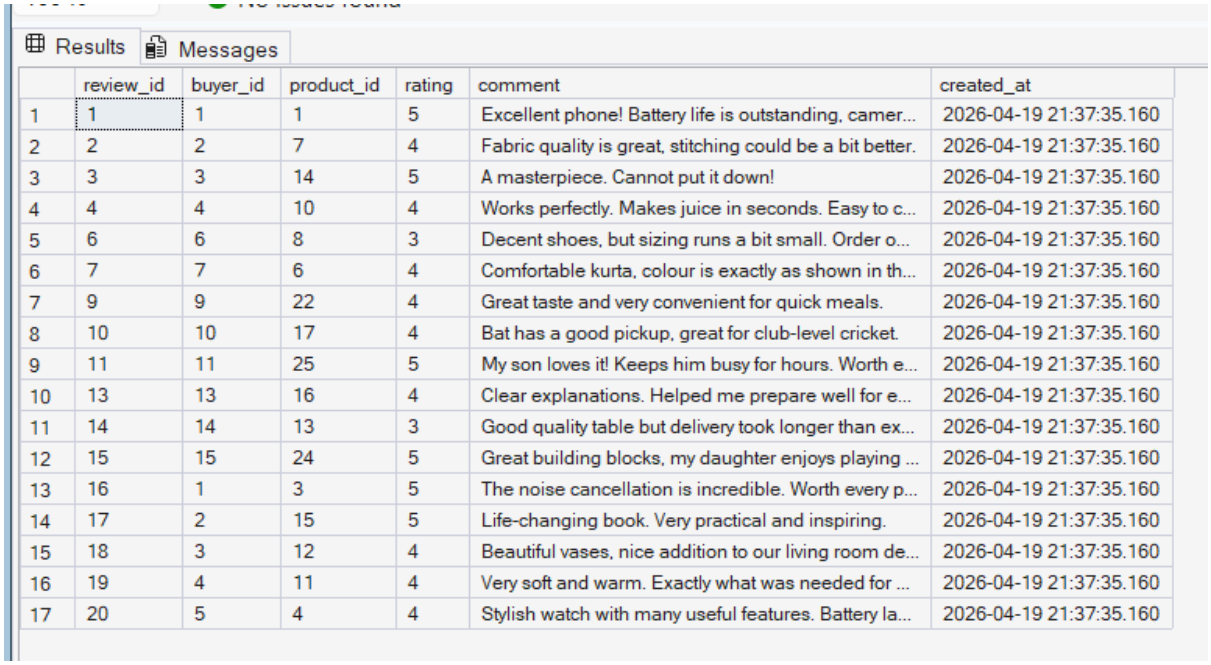
```
-- Delete a review (moderation action)
DELETE FROM Review WHERE review_id = 5;
```

Output of Delete:



Results Messages

| | product_id | seller_id | subcategory_id | name | price | stock | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 16 | 1 | Samsung Galaxy A55 5G | 95000.00 | 100 | active |
| 2 | 2 | 16 | 2 | HP Pavilion 15 Laptop i5 | 149999.00 | 45 | active |



Results Messages

| | review_id | buyer_id | product_id | rating | comment | created_at |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 5 | Excellent phone! Battery life is outstanding, camer... | 2026-04-19 21:37:35.160 |
| 2 | 2 | 2 | 7 | 4 | Fabric quality is great, stitching could be a bit better. | 2026-04-19 21:37:35.160 |
| 3 | 3 | 3 | 14 | 5 | A masterpiece. Cannot put it down! | 2026-04-19 21:37:35.160 |
| 4 | 4 | 4 | 10 | 4 | Works perfectly. Makes juice in seconds. Easy to c... | 2026-04-19 21:37:35.160 |
| 5 | 6 | 6 | 8 | 3 | Decent shoes, but sizing runs a bit small. Order o... | 2026-04-19 21:37:35.160 |
| 6 | 7 | 7 | 6 | 4 | Comfortable kurta, colour is exactly as shown in th... | 2026-04-19 21:37:35.160 |
| 7 | 9 | 9 | 22 | 4 | Great taste and very convenient for quick meals. | 2026-04-19 21:37:35.160 |
| 8 | 10 | 10 | 17 | 4 | Bat has a good pickup, great for club-level cricket. | 2026-04-19 21:37:35.160 |
| 9 | 11 | 11 | 25 | 5 | My son loves it! Keeps him busy for hours. Worth e... | 2026-04-19 21:37:35.160 |
| 10 | 13 | 13 | 16 | 4 | Clear explanations. Helped me prepare well for e... | 2026-04-19 21:37:35.160 |
| 11 | 14 | 14 | 13 | 3 | Good quality table but delivery took longer than ex... | 2026-04-19 21:37:35.160 |
| 12 | 15 | 15 | 24 | 5 | Great building blocks, my daughter enjoys playing ... | 2026-04-19 21:37:35.160 |
| 13 | 16 | 1 | 3 | 5 | The noise cancellation is incredible. Worth every p... | 2026-04-19 21:37:35.160 |
| 14 | 17 | 2 | 15 | 5 | Life-changing book. Very practical and inspiring. | 2026-04-19 21:37:35.160 |
| 15 | 18 | 3 | 12 | 4 | Beautiful vases, nice addition to our living room de... | 2026-04-19 21:37:35.160 |
| 16 | 19 | 4 | 11 | 4 | Very soft and warm. Exactly what was needed for ... | 2026-04-19 21:37:35.160 |
| 17 | 20 | 5 | 4 | 4 | Stylish watch with many useful features. Battery la... | 2026-04-19 21:37:35.160 |

# 10. Analytical Queries

## 10.1 Top-Performing Entities

### Query 1 - Top 5 Buyers by Total Spending

```
SELECT TOP 5
   u.user_id, u.name,
   SUM(o.total_amount) AS total_spent
FROM [User] u
JOIN [Order] o ON u.user_id = o.buyer_id
GROUP BY u.user_id, u.name
ORDER BY total_spent DESC;
```

### Query 2 - Top 5 Best-Selling Products

```
SELECT TOP 5
   p.product_id, p.name,
   SUM(oi.quantity) AS total_units_sold,
   SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM Product p
JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.name
ORDER BY total_units_sold DESC;
```

### Query 3 - Top 5 Sellers by Revenue

```
SELECT TOP 5
   s.user_id, s.shop_name,
   SUM(oi.quantity * oi.unit_price) AS total_sales
FROM Seller s
JOIN Product p ON s.user_id = p.seller_id
JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY s.user_id, s.shop_name
ORDER BY total_sales DESC;
```

## 10.2 Trend Detection

### Query 4 - Monthly Sales Trend

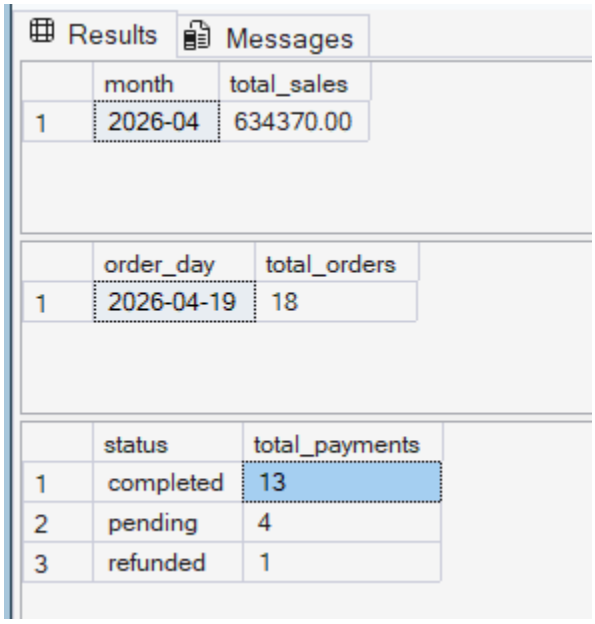
```
SELECT
    FORMAT(order_date, 'yyyy-MM') AS sale_month,
    COUNT(*) AS total_orders,
    SUM(total_amount) AS monthly_revenue
FROM [Order]
WHERE status NOT IN ('cancelled')
GROUP BY FORMAT(order_date, 'yyyy-MM')
ORDER BY sale_month;
```

### Query 5 - Daily Order Activity

```
SELECT
    CAST(order_date AS DATE) AS order_day,
    COUNT(*) AS total_orders
FROM [Order]
GROUP BY CAST(order_date AS DATE)
ORDER BY order_day;
```

### Query 6 - Payment Method Distribution

```
SELECT
    method,
    COUNT(*) AS usage_count,
    SUM(amount) AS total_amount_processed
FROM Payment
WHERE status = 'completed'
GROUP BY method
ORDER BY usage_count DESC;
```



## 10.3 Inactive / Exceptional Cases

### Query 7 - Buyers with No Orders

```
SELECT u.user_id, u.name, u.email
FROM [User] u
INNER JOIN Buyer b ON u.user_id = b.user_id
LEFT JOIN [Order] o ON u.user_id = o.buyer_id
WHERE o.order_id IS NULL;
```

### Query 8 - Products Never Purchased

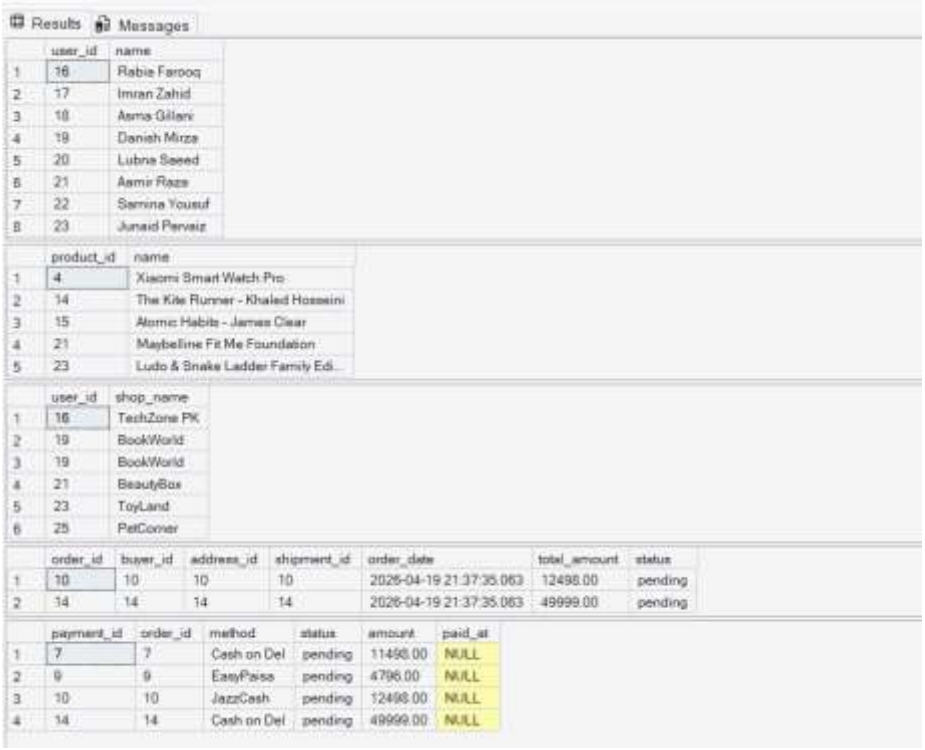
```
SELECT p.product_id, p.name, p.price, p.stock
FROM Product p
LEFT JOIN OrderItem oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL AND p.status = 'active';
```

### Query 9 - Sellers with No Sales

```
SELECT s.user_id, s.shop_name
FROM Seller s
LEFT JOIN Product p ON s.user_id = p.seller_id
LEFT JOIN OrderItem oi ON p.product_id = oi.product_id
WHERE oi.product_id IS NULL;
```

### Query 10 - Pending and Failed Payments

```
SELECT p.payment_id, p.order_id, p.method, p.status, p.amount
FROM Payment p
WHERE p.status IN ('pending', 'failed')
ORDER BY p.payment_id DESC;
```



Results Messages

| | user_id | name |
|---|---|---|
| 1 | 16 | Rabia Farooq |
| 2 | 17 | Imran Zahid |
| 3 | 18 | Asma Gillani |
| 4 | 19 | Danish Mirza |
| 5 | 20 | Lubna Saeed |
| 6 | 21 | Aamir Raza |
| 7 | 22 | Samina Yousuf |
| 8 | 23 | Junaid Pervaiz |

| | product_id | name |
|---|---|---|
| 1 | 4 | Xiaomi Smart Watch Pro |
| 2 | 14 | The Kite Runner - Khaled Hosseini |
| 3 | 15 | Atomic Habits - James Clear |
| 4 | 21 | Maybelline Fit Me Foundation |
| 5 | 23 | Ludo & Snake Ladder Family Edi... |

| | user_id | shop_name |
|---|---|---|
| 1 | 16 | TechZone PK |
| 2 | 19 | BookWorld |
| 3 | 19 | BookWorld |
| 4 | 21 | BeautyBox |
| 5 | 23 | ToyLand |
| 6 | 25 | PetCorner |

| | order_id | buyer_id | address_id | shipment_id | order_date | total_amount | status |
|---|---|---|---|---|---|---|---|
| 1 | 10 | 10 | 10 | 10 | 2026-04-19 21:37:35.063 | 12498.00 | pending |
| 2 | 14 | 14 | 14 | 14 | 2026-04-19 21:37:35.063 | 49999.00 | pending |

| | payment_id | order_id | method | status | amount | paid_at |
|---|---|---|---|---|---|---|
| 1 | 7 | 7 | Cash on Del | pending | 11498.00 | NULL |
| 2 | 9 | 9 | EasyPaisa | pending | 4796.00 | NULL |
| 3 | 10 | 10 | JazzCash | pending | 12498.00 | NULL |
| 4 | 14 | 14 | Cash on Del | pending | 49999.00 | NULL |

# 11. Advanced SQL - JOINs, Subqueries, GROUP BY, HAVING

## 11.1 Multi-Table JOINs

### Query 11 - Full Order Details (5-Table JOIN)

```
SELECT
    o.order_id, u.name AS buyer_name,
    p.name AS product_name, oi.quantity, oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total,
    o.status AS order_status, sh.status AS shipment_status
FROM [Order] o
JOIN [User] u ON o.buyer_id = u.user_id
JOIN OrderItem oi ON o.order_id = oi.order_id
JOIN Product p ON oi.product_id = p.product_id
JOIN Shipment sh ON o.shipment_id = sh.shipment_id
ORDER BY o.order_id, oi.product_id;
```

| | order_id | order_date | total_amount | payment_id | method | payment_status | shipment_id | shipment_status |
|---|---|---|---|---|---|---|---|---|
| 1 | 1 | 2026-04-19 21:37:35.063 | 89999.00 | 1 | Credit Card | completed | 1 | delivered |
| 2 | 2 | 2026-04-19 21:37:35.063 | 5998.00 | 2 | EasyPaisa | completed | 2 | delivered |
| 3 | 4 | 2026-04-19 21:37:35.063 | 12498.00 | 4 | Credit Card | completed | 4 | delivered |
| 4 | 5 | 2026-04-19 21:37:35.063 | 13498.00 | 5 | JazzCash | completed | 5 | delivered |
| 5 | 6 | 2026-04-19 21:37:35.063 | 5498.00 | 6 | Debit Card | completed | 6 | delivered |
| 6 | 7 | 2026-04-19 21:37:35.063 | 11498.00 | 7 | Cash on ... | pending | 7 | in_transit |
| 7 | 8 | 2026-04-19 21:37:35.063 | 149999.00 | 8 | Credit Card | completed | 8 | dispatched |
| 8 | 9 | 2026-04-19 21:37:35.063 | 4796.00 | 9 | EasyPaisa | pending | 9 | pending |

### Query 12 - Revenue by Category (4-Table JOIN)

```
SELECT
    c.name AS category,
    COUNT(DISTINCT oi.order_id) AS total_orders,
    SUM(oi.quantity * oi.unit_price) AS category_revenue
FROM Category c
JOIN Subcategory sc ON c.category_id = sc.category_id
JOIN Product p ON sc.subcategory_id = p.subcategory_id
JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY c.name
ORDER BY category_revenue DESC;
```

## 11.2 Nested Subqueries

### Query 13 - Products Above Average Price in Their Category

```
SELECT p.product_id, p.name, p.price, sc.name AS subcategory
FROM Product p
JOIN Subcategory sc ON p.subcategory_id = sc.subcategory_id
WHERE p.price > (
    SELECT AVG(p2.price)
    FROM Product p2
    WHERE p2.subcategory_id = p.subcategory_id
      AND p2.status = 'active'
);
```

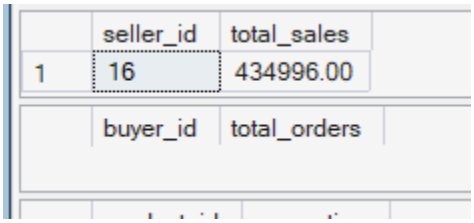
### Query 14 - Buyers Who Have Spent More Than the Average

```
SELECT u.user_id, u.name, SUM(o.total_amount) AS total_spent
FROM [User] u
JOIN [Order] o ON u.user_id = o.buyer_id
GROUP BY u.user_id, u.name
HAVING SUM(o.total_amount) > (
   SELECT AVG(total_amount) FROM [Order]
);
```

## 11.3 GROUP BY & HAVING

### Query 15 - High-Performing Sellers (Revenue > 50,000)

```
SELECT
   s.user_id, s.shop_name,
   SUM(oi.quantity * oi.unit_price) AS total_sales,
   COUNT(DISTINCT oi.order_id) AS total_orders
FROM Seller s
JOIN Product p ON s.user_id = p.seller_id
JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY s.user_id, s.shop_name
HAVING SUM(oi.quantity * oi.unit_price) > 50000
ORDER BY total_sales DESC;
```



| | seller_id | total_sales |
|---|---|---|
| 1 | 16 | 434996.00 |

| | buyer_id | total_orders |
|---|---|---|

### Query 16 - Products with Average Rating >= 4 and More Than 5 Reviews

```
SELECT
    p.product_id, p.name,
    AVG(CAST(r.rating AS DECIMAL(4,2))) AS avg_rating,
    COUNT(r.review_id) AS review_count
FROM Product p
JOIN Review r ON p.product_id = r.product_id
GROUP BY p.product_id, p.name
HAVING AVG(CAST(r.rating AS DECIMAL(4,2))) >= 4.0
   AND COUNT(r.review_id) > 5
ORDER BY avg_rating DESC;
```

| | product_id | avg_rating |
|---|---|---|
| 4 | 6 | 4 |
| 5 | 7 | 4 |
| 6 | 10 | 4 |
| 7 | 11 | 4 |
| 8 | 15 | 5 |
| 9 | 16 | 4 |
| 10 | 17 | 4 |
| 11 | 22 | 4 |

## 11.4 Complex Logical Conditions

### Query 17 - Low Stock Active Products Across All Warehouses

```
SELECT
    p.product_id, p.name, p.stock AS catalog_stock,
    i.warehouse_id, i.quantity AS warehouse_qty, i.min_threshold
FROM Product p
JOIN Inventory i ON p.product_id = i.product_id
WHERE p.status = 'active'
  AND i.quantity < i.min_threshold
  AND p.stock < 20
ORDER BY p.stock ASC;
```

| | order_id | buyer_id | address_id | shipment_id | order_date | total_amount | status |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 2026-04-19 21:37:35.063 | 89999.00 | delivered |
| 2 | 8 | 8 | 8 | 8 | 2026-04-19 21:37:35.063 | 149999.00 | shipped |
| 3 | 12 | 12 | 12 | 12 | 2026-04-19 21:37:35.063 | 119999.00 | delivered |
| 4 | 16 | 1 | 21 | 16 | 2026-04-19 21:37:35.063 | 74999.00 | delivered |

# 12. Practical Enhancements

The following enhancements were implemented to improve query performance, provide abstraction, automate business logic, and enforce data integrity beyond basic constraints.

## 12.1 Indexes

Justification: Indexes are created on high-frequency lookup and join columns to reduce full table scans and significantly improve query performance on the most commonly executed operations.

```
-- Index on Product seller_id (frequently joined to Seller)
CREATE INDEX idx_product_seller_id ON Product(seller_id);
```

```
-- Index on Product subcategory_id (used in category browsing)
CREATE INDEX idx_product_subcategory_id ON Product(subcategory_id);
```

```
-- Index on Order buyer_id (order history lookups)
CREATE INDEX idx_order_buyer_id ON [Order](buyer_id);
```

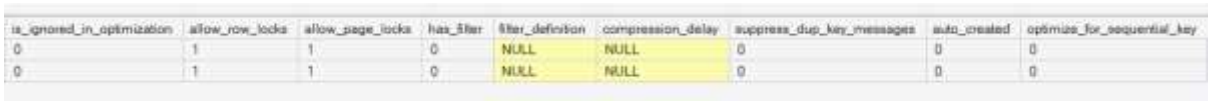
```
-- Index on OrderItem product_id (sales analytics)
CREATE INDEX idx_orderitem_product_id ON OrderItem(product_id);
```

```
-- Composite index on Review (buyer_id, product_id)
CREATE INDEX idx_review_buyer_product ON Review(buyer_id, product_id);
```

```
-- Index on Inventory product_id (stock lookups)
CREATE INDEX idx_inventory_product_id ON Inventory(product_id);
```

```
-- Verify indexes on Product table
SELECT * FROM sys.indexes WHERE object_id = OBJECT_ID('Product');
```

| object_id | name | index_id | type | type_desc | is_unique | data_space_id | ignore_dup_key | is_primary_key | is_unique_constraint | fill_factor | is_padded | is_disabled | is_hypothetical |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | PK__Product__47027DF502E7ED2E | 1 | 1 | CLUSTERED | 1 | 1 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 2 | idx_product_id | 6 | 2 | NONCLUSTERED | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |



| is_ignored_in_optimization | allow_row_locks | allow_page_locks | has_filter | filter_definition | compression_delay | suppress_dup_key_messages | auto_created | optimize_for_sequential_key |
|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | NULL | NULL | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 | NULL | NULL | 0 | 0 | 0 |

## 12.2 Views

Justification: Views provide a simplified abstraction layer over complex multi-table queries, enabling the frontend and reporting layer to retrieve pre-joined, ready-to-use data without repeating complex SQL logic.

### View 1 - ProductCatalog

```
CREATE VIEW ProductCatalog AS
SELECT
    p.product_id, p.name AS product_name, p.price, p.stock, p.status,
    s.shop_name AS seller_name,
    sc.name AS subcategory, c.name AS category,
    AVG(CAST(r.rating AS DECIMAL(4,2))) AS avg_rating,
    COUNT(r.review_id) AS review_count
FROM Product p
JOIN Seller s ON p.seller_id = s.user_id
JOIN Subcategory sc ON p.subcategory_id = sc.subcategory_id
JOIN Category c ON sc.category_id = c.category_id
LEFT JOIN Review r ON p.product_id = r.product_id
WHERE p.status = 'active'
GROUP BY p.product_id, p.name, p.price, p.stock, p.status, s.shop_name, sc.name, c.name;
```

| | product_id | product_name | price | stock | shop_name | subcategory_name |
|---|---|---|---|---|---|---|
| 1 | 1 | Samsung Galaxy A55 5G | 95000.00 | 100 | TechZone PK | Mobile Phones |
| 2 | 2 | HP Pavilion 15 Laptop i5 | 149999.00 | 45 | TechZone PK | Laptops |
| 3 | 3 | Sony WH-1000XM5 Headphones | 74999.00 | 60 | TechZone PK | Headphones |
| 4 | 4 | Xiaomi Smart Watch Pro | 19999.00 | 90 | TechZone PK | Smartwatches |
| 5 | 5 | Canon EOS 1500D DSLR | 119999.00 | 20 | TechZone PK | Cameras |
| 6 | 6 | Men Linen Kurta - Navy Blue | 3499.00 | 200 | FashionHub | Men Clothing |
| 7 | 7 | Women Lawn Suit - Floral Print | 2999.00 | 350 | FashionHub | Women Clothing |
| 8 | 8 | Running Shoes - White (Men) | 6999.00 | 150 | FashionHub | Footwear |

### View 2 - OrderSummary

```
CREATE VIEW OrderSummary AS
SELECT
    o.order_id, u.name AS buyer_name, u.email,
    o.order_date, o.total_amount, o.status AS order_status,
    sh.status AS shipment_status, sh.tracking_num,
    ca.name AS carrier_name,
    pay.method, pay.status AS payment_status
FROM [Order] o
JOIN [User] u ON o.buyer_id = u.user_id
JOIN Shipment sh ON o.shipment_id = sh.shipment_id
JOIN Carrier ca ON sh.carrier_id = ca.carrier_id
LEFT JOIN Payment pay ON o.order_id = pay.order_id;
```

| | order_id | buyer_id | order_date | total_amount | order_status | shipment_status |
|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 2026-04-19 21:37:35.063 | 89999.00 | delivered | delivered |
| 2 | 2 | 2 | 2026-04-19 21:37:35.063 | 5998.00 | delivered | delivered |
| 3 | 4 | 4 | 2026-04-19 21:37:35.063 | 12498.00 | delivered | delivered |
| 4 | 5 | 5 | 2026-04-19 21:37:35.063 | 13498.00 | delivered | delivered |
| 5 | 6 | 6 | 2026-04-19 21:37:35.063 | 5498.00 | delivered | delivered |
| 6 | 11 | 11 | 2026-04-19 21:37:35.063 | 42999.00 | delivered | delivered |
| 7 | 12 | 12 | 2026-04-19 21:37:35.063 | 119999.00 | delivered | delivered |
| 8 | 15 | 15 | 2026-04-19 21:37:35.063 | 4998.00 | delivered | delivered |

### View 3 - SellerDashboard

```
CREATE VIEW SellerDashboard AS
SELECT
   s.user_id AS seller_id, s.shop_name,
   COUNT(DISTINCT p.product_id) AS total_products,
   SUM(oi.quantity) AS total_units_sold,
   SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM Seller s
LEFT JOIN Product p ON s.user_id = p.seller_id
LEFT JOIN OrderItem oi ON p.product_id = oi.product_id
GROUP BY s.user_id, s.shop_name;
```

| | seller_id | shop_name | total_products | total_sales |
|---|---|---|---|---|
| 1 | 16 | TechZone PK | 5 | 434996.00 |
| 2 | 18 | HomeNest | 4 | 70494.00 |
| 3 | 24 | AutoParts Pl... | 1 | 42999.00 |
| 4 | 17 | FashionHub | 4 | 27993.00 |
| 5 | 20 | SportsKing | 3 | 25996.00 |
| 6 | 23 | ToyLand | 2 | 4998.00 |
| 7 | 22 | GroceryDirect | 1 | 4796.00 |
| 8 | 19 | BookWorld | 3 | 3598.00 |

## 12.3 Triggers

Justification: Triggers enforce automated business logic at the database level, ensuring data consistency without relying on application-layer enforcement.

### Trigger 1 - trg_reduce_stock (AFTER INSERT on OrderItem)

```
CREATE TRIGGER trg_reduce_stock
ON OrderItem
AFTER INSERT
AS
BEGIN
   SET NOCOUNT ON;
   UPDATE Product
   SET stock = stock - i.quantity
   FROM Product p
   INNER JOIN inserted i ON p.product_id = i.product_id;
END;
```

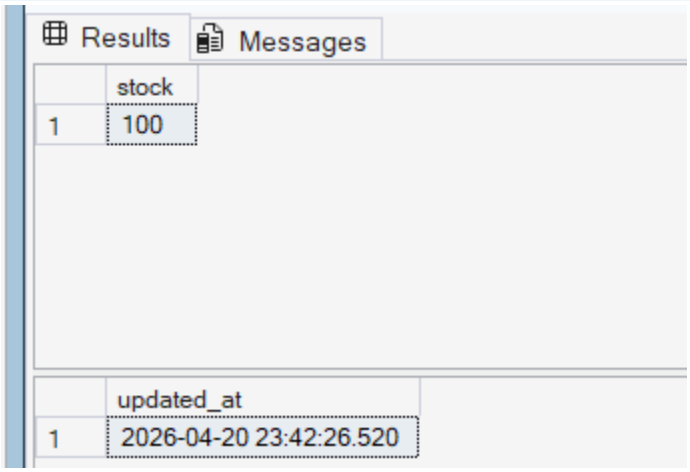
### Trigger 2 - trg_prevent_negative_stock (AFTER UPDATE on Product)

```
CREATE TRIGGER trg_prevent_negative_stock
ON Product
AFTER UPDATE
AS
BEGIN
   IF EXISTS (SELECT 1 FROM inserted WHERE stock < 0)
   BEGIN
      RAISERROR('Stock cannot go below zero.', 16, 1);
```

```
        ROLLBACK TRANSACTION;
    END
END;
```

### Trigger 3 - trg_update_cart_timestamp (AFTER INSERT/UPDATE/DELETE on CartItem)

```
CREATE TRIGGER trg_update_cart_timestamp
ON CartItem
AFTER INSERT, UPDATE, DELETE
AS
BEGIN
    SET NOCOUNT ON;
    UPDATE Cart SET updated_at = GETDATE()
    WHERE cart_id IN (
        SELECT cart_id FROM inserted
        UNION
        SELECT cart_id FROM deleted
    );
END;
```

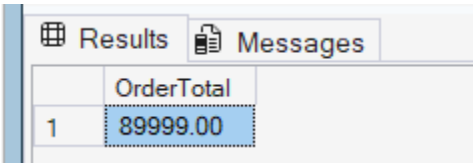


## 12.4 User-Defined Functions

### Function 1 - fn_CalculateOrderTotal

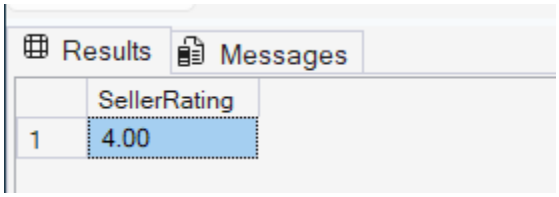
```
CREATE FUNCTION fn_CalculateOrderTotal (@order_id INT)
RETURNS DECIMAL(12,2)
AS BEGIN
   DECLARE @total DECIMAL(12,2);
   SELECT @total = SUM(quantity * unit_price)
   FROM OrderItem WHERE order_id = @order_id;
   RETURN ISNULL(@total, 0.00);
END;
```

```
-- Usage: SELECT dbo.fn_CalculateOrderTotal(1) AS total;
```




### Function 2 - fn_SellerRating

```
CREATE FUNCTION fn_SellerRating (@seller_id INT)
RETURNS DECIMAL(3,2)
AS BEGIN
   DECLARE @rating DECIMAL(3,2);
   SELECT @rating = AVG(CAST(r.rating AS DECIMAL(4,2)))
   FROM Review r
   JOIN Product p ON r.product_id = p.product_id
   WHERE p.seller_id = @seller_id;
   RETURN ISNULL(@rating, 0.00);
END;
```



# 13. Transactions & Error Handling

Transactions ensure atomicity for multi-step operations. If any step fails, the entire transaction is rolled back to maintain database consistency. The example below demonstrates a complete order placement workflow.

## 13.1 Place Order Transaction (with TRY-CATCH)

```
BEGIN TRANSACTION;

BEGIN TRY
   -- Step 1: Create a Shipment record first (required FK for Order)
   INSERT INTO Shipment (carrier_id, status)
   VALUES (1, 'pending');
   DECLARE @shipment_id INT = SCOPE_IDENTITY();

   -- Step 2: Insert the Order
   INSERT INTO [Order] (buyer_id, address_id, shipment_id, total_amount, status)
   VALUES (1, 1, @shipment_id, 0, 'pending');
   DECLARE @order_id INT = SCOPE_IDENTITY();

   -- Step 3: Insert OrderItems (triggers trg_reduce_stock automatically)
   INSERT INTO OrderItem (order_id, product_id, quantity, unit_price)
   VALUES (@order_id, 1, 1, 62999.00);

   INSERT INTO OrderItem (order_id, product_id, quantity, unit_price)
   VALUES (@order_id, 3, 2, 4999.00);

   -- Step 4: Update Order total_amount using the UDF
   UPDATE [Order]
   SET total_amount = dbo.fn_CalculateOrderTotal(@order_id)
   WHERE order_id = @order_id;

   -- Step 5: Create Buyer_Order junction entry
   INSERT INTO Buyer_Order (buyer_id, order_id) VALUES (1, @order_id);

   -- Step 6: Create Payment record
   INSERT INTO Payment (order_id, method, status, amount)
   VALUES (@order_id, 'credit_card', 'pending',
         dbo.fn_CalculateOrderTotal(@order_id));

   COMMIT TRANSACTION;
   PRINT 'Order placed successfully. Order ID: ' + CAST(@order_id AS VARCHAR);

END TRY
BEGIN CATCH
   ROLLBACK TRANSACTION;
   PRINT 'Transaction Failed: ' + ERROR_MESSAGE();
END CATCH;
GO
```

## 13.2 Verify Transaction Results

```
SELECT * FROM [Order]   ORDER BY order_id DESC;
SELECT * FROM OrderItem ORDER BY order_id DESC;
SELECT stock FROM Product WHERE product_id IN (1, 3);
SELECT * FROM Payment   ORDER BY payment_id DESC;
```

Results | Messages

| | order_id | buyer_id | address_id | shipment_id | order_date | total_amount | status |
|---|---|---|---|---|---|---|---|
| 1 | 20 | 5 | 25 | 20 | 2026-04-19 21:37:35.063 | 3499.00 | confirmed |
| 2 | 19 | 4 | 24 | 19 | 2026-04-19 21:37:35.063 | 19999.00 | shipped |
| 3 | 17 | 2 | 22 | 17 | 2026-04-19 21:37:35.063 | 7998.00 | returned |
| 4 | 16 | 1 | 21 | 16 | 2026-04-19 21:37:35.063 | 74999.00 | delivered |
| 5 | 15 | 15 | 15 | 15 | 2026-04-19 21:37:35.063 | 4998.00 | delivered |
| 6 | 14 | 14 | 14 | 14 | 2026-04-19 21:37:35.063 | 49999.00 | pending |
| 7 | 13 | 13 | 13 | 13 | 2026-04-19 21:37:35.063 | 3598.00 | shipped |
| 8 | 12 | 12 | 12 | 12 | 2026-04-19 21:37:35.063 | 119999.00 | delivered |

| | order_id | product_id | quantity | unit_price |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 89999.00 |
| 2 | 2 | 7 | 2 | 2999.00 |
| 3 | 4 | 10 | 1 | 4999.00 |
| 4 | 4 | 12 | 3 | 2499.00 |
| 5 | 5 | 18 | 1 | 3499.00 |
| 6 | 5 | 19 | 1 | 9999.00 |
| 7 | 6 | 8 | 1 | 6999.00 |
| 8 | 6 | 20 | 1 | 3499.00 |

| | stock |
|---|---|
| 1 | 100 |

# 14. Data Integrity & Validation Summary

## 14.1 Constraint Coverage

| Constraint Type | Tables Applied | Example |
|---|---|---|
| PRIMARY KEY | All 21 tables | user_id INT IDENTITY(1,1) PRIMARY KEY |
| FOREIGN KEY | 17 tables | FOREIGN KEY (seller_id) REFERENCES Seller(user_id) |
| UNIQUE | User, Cart, Payment, Order, Review, Inventory, Buyer_Order | UNIQUE (buyer_id, product_id) - one review per product |
| NOT NULL | All core attribute columns | email VARCHAR(150) NOT NULL |
| CHECK (Range) | Product, Buyer, Review, Payment, Inventory | CHECK (price > 0); CHECK (rating BETWEEN 1 AND 5) |
| CHECK (ENUM) | Product, Order, Payment, Shipment, Transaction | CHECK (status IN ('pending','confirmed','shipped','delivered','cancelled','returned')) |
| DEFAULT | All timestamp & status columns | DEFAULT GETDATE(); DEFAULT 'pending'; DEFAULT 0 |
| CASCADE DELETE | Buyer, Seller, Address, CartItem, ProductImage, ProductAttribute, Transaction | ON DELETE CASCADE - deleting a User removes all child records |
| TRIGGER | Product, CartItem | trg_reduce_stock, trg_prevent_negative_stock, trg_update_cart_timestamp |

# 15. Front-End Interface

## 15.1 Frontend Interface Overview

MarketHub is a full-stack web marketplace application built using Flask as the backend framework with Jinja2 templating for dynamic frontend rendering. The frontend consists of 11 HTML templates that share a common base layout, providing a consistent look and feel across all pages. The application supports three distinct user roles — Buyer, Seller, and Admin — each with a dedicated interface and navigation flow.

### 15.1.1 Technology Stack

| UI Element | Description / Functionality |
|---|---|
| Templating Engine | Jinja2 (Flask) — all pages extend base.html via {% extends %} / {% block content %} |
| Styling | Custom CSS3 embedded in base.html — dark navy (#1A1A2E) theme with red (#E94560) accent color |
| Typography | Segoe UI system font for clean, readable interface |
| JavaScript | Vanilla JS used for dynamic role-based form toggling on the Register page |
| Layout | CSS Grid and Flexbox for responsive product cards, stat panels, and forms |
| Backend | Python 3 / Flask — routes in app.py serve all templates with context data |
| Database | SQL Server connected via pyodbc — all UI data is live from MarketHubDB |

### 15.1.2 Template Structure

| UI Element | Description / Functionality |
|---|---|
| base.html | Master layout — navbar, flash messages, container wrapper, footer. All other templates extend this. |
| login.html | User authentication — email/password form |
| register.html | New account creation — dynamic fields based on role (Buyer/Seller) |
| buyer_home.html | Product browsing — search, category filter, product grid |
| cart.html | Shopping cart — item table with quantities and totals |
| place_order.html | Checkout — address selection/creation, payment method selection |
| my_orders.html | Order history — table with status and payment badges |
| review.html | Product review form — star rating + comment |
| seller_dashboard.html | Seller control panel — stats cards + product management table |
| add_product.html | New product listing form |
| edit_product.html | Product update form — pre-filled with existing data |
| admin_dashboard.html | Admin analytics — platform-wide stats, top products, revenue chart |

## 15.2 Base Layout & Navigation(base.html)

All pages in MarketHub inherit from base.html, which defines the global structure: navbar, flash message area, main content container, and footer. This ensures a consistent user experience across every page of the application.

### 15.2.1 Navbar

The navigation bar is sticky (stays at top on scroll) and dynamically changes its links based on the logged-in user's role, using Jinja2 session checks.

| UI Element | Description / Functionality |
|---|---|
| Logo | 🛒 MarketHub — always visible, links to home |
| Guest (not logged in) | Login link + 'Register' pill button in red |
| Buyer navbar | Displays: Browse \| 🛒 Cart \| My Orders \| Logout button |
| Seller navbar | Displays: Dashboard \| + Add Product \| Logout button |
| Admin navbar | Displays: Reports \| Logout button |
| User pill | Shows 'Hi, [Name] ([role])' when logged in |


*Figure 4:Navbar-Guest State*


*Figure 5:Navbar - Buyer State*


*Figure 6:Navbar - Seller State*

### 15.2.2 Flash Messages

Flash messages appear below the navbar and above the page content. They use color-coded alerts to provide instant feedback to the user after actions such as login, registration, order placement, or product updates.

| UI Element | Description / Functionality |
|---|---|
| Success (green) | Registration successful, item added to cart, order placed, review submitted |
| Danger (red) | Invalid login credentials, duplicate email, database errors |
| Warning (yellow) | Empty cart on checkout attempt |
| Info (blue) | Product removed / soft-deleted |


*Figure 7:Flash message -Item added to cart successfully*


*Figure 8:Flash message - Order Placed successfully*


*Figure 9:Flash message - Invalid credentials*

## 15.3 Authentication Pages

### 15.3.1 Login page

The login page is the entry point of the application — the root URL '/' redirects directly to '/login'. It provides a clean, centered card layout with email and password fields.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /login |
| Layout | Centered card — max-width 420px, vertically padded for visual balance |
| Fields | Email Address (type=email, required) \| Password (type=password, required) |
| Submit Button | 'Login' — full-width red primary button |
| Register Link | Redirects to /register — shown below the form |
| Role Detection | After login, Flask checks if user_id exists in Buyer, Seller, or neither (Admin) table and redirects accordingly |
| Error Handling | Wrong credentials flash a red danger message; form stays on screen |

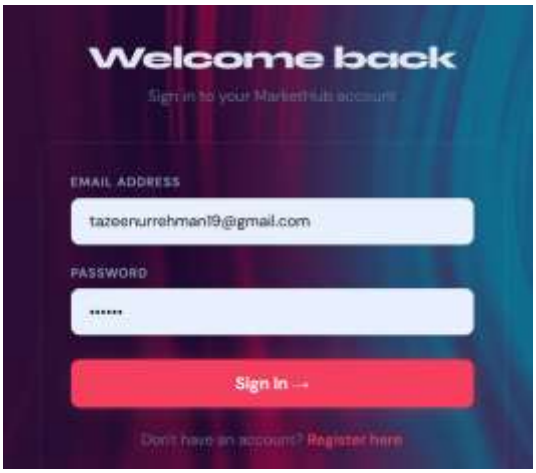



*Figure 10:Login Page-Credentials form*

### 15.3.2 Register page

The registration page allows new users to sign up as either a Buyer or a Seller. A key frontend feature is the dynamic role toggle: when the user changes the 'Register as' dropdown, JavaScript shows or hides role-specific fields without a page reload.

| UI Element | Description / Functionality |
|---|---|
| **Page URL** | /register |
| **Layout** | Centered card — max-width 480px |
| **Common Fields** | Full Name \| Email Address \| Password \| Phone Number |
| **Role Dropdown** | Buyer / Seller — triggers JS toggleFields() on change |
| **Buyer-only Field** | Date of Birth (with validation note: must be 16+) |
| **Seller-only Field** | Shop Name — revealed when 'Seller' is selected in dropdown |
| **JavaScript** | toggleFields() — shows/hides #buyerFields and #sellerFields divs dynamically |
| **Submit** | 'Create Account' — full-width red button |
| **Login Link** | Shown below for existing users |


*Figure 11:Register Page - Buyer Mode*


*Figure 12: Register Page - Seller Mode*

## 15.4 Buyer Interface

The Buyer interface provides the full shopping experience: browsing products, managing a cart, placing orders, tracking order history, and submitting product reviews. Buyers are redirected to /buyer immediately after logging in.

### 15.4.1 Product Browse Page

This is the main marketplace page where buyers discover products. All active products from the database are displayed in a responsive grid, with real-time search and category filtering.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /buyer |
| Search Bar | Text input — searches product names using SQL LIKE query (%keyword%) |
| Category Filter | Dropdown — dynamically populated from the Category table; filters by category name |
| Search + Clear Buttons | 'Search' submits GET form \| 'Clear' resets to show all products |
| Product Grid | CSS Grid — auto-fill, minmax(220px) — responsive across screen sizes |
| Product Card | Shows: Product name \| Price (Rs) \| Stock count \| Category \| Shop name |
| Add to Cart | Red primary button — calls /add_to_cart/<product_id> route |
| Review Button | Blue info button — navigates to /review/<product_id> |
| Empty State | Shows ' No products found.' message with 'Show All' button when no results match |

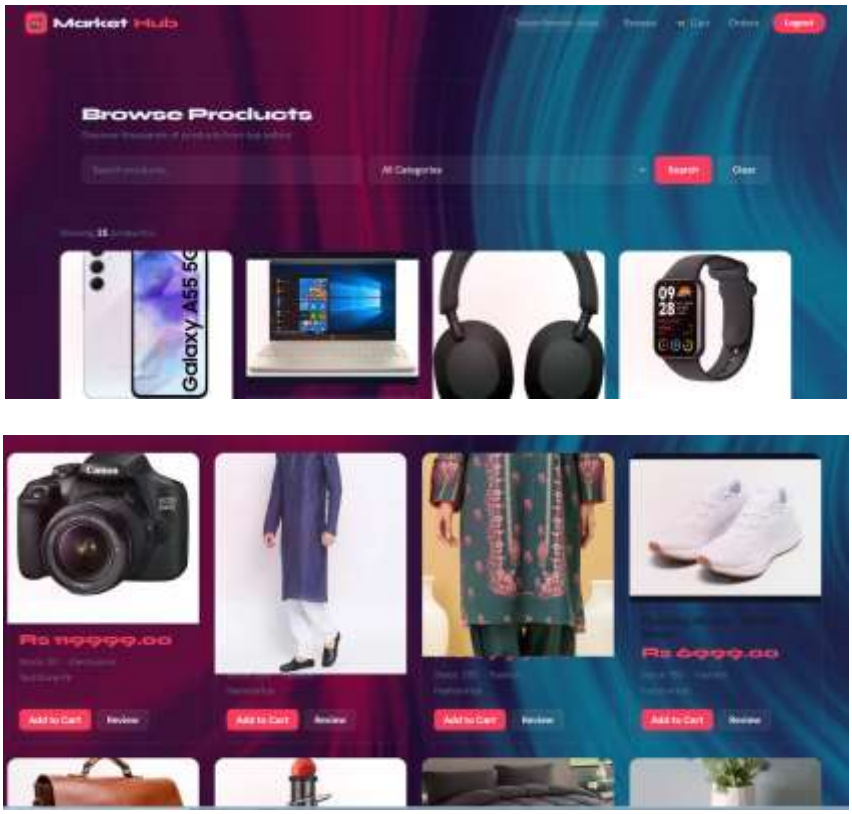



*Figure 13:Buyer Home - Full Product Grid*

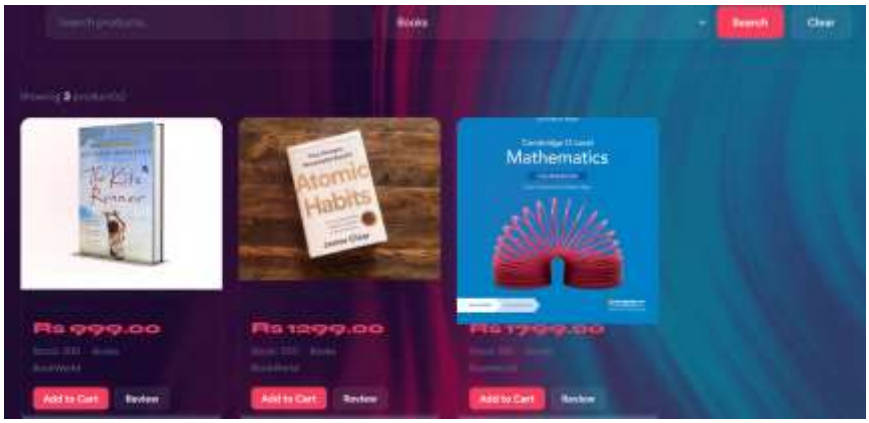



*Figure 14:Buyer Home - Search/Filter in Action*

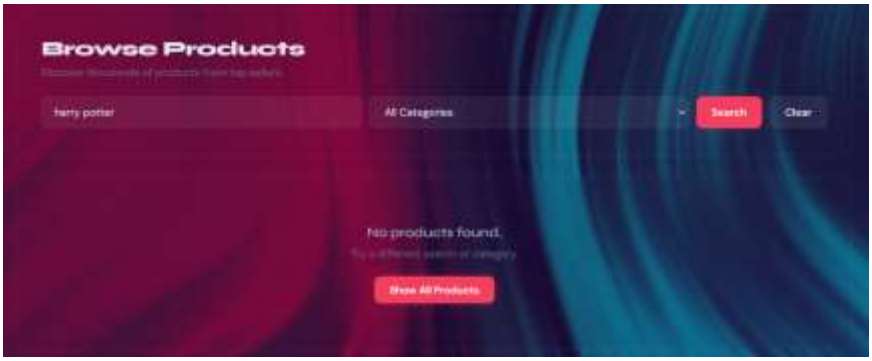



*Figure 15:Buyer - Empty Search Result*

### 15.4.2 Shopping Cart

The cart page shows all items the buyer has added, with quantities, individual prices, subtotals, and the order total. It is the gateway to the checkout process.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /cart |
| Items Table | Columns: Product \| Unit Price \| Quantity \| Subtotal |
| Total Display | Calculates sum of all subtotals — shown right-aligned below the table in red |
| Continue Shopping | Secondary button — returns to /buyer |
| Checkout Button | Red primary button — navigates to /place_order |
| Empty Cart State | Shows '🛒 Your cart is empty!' with 'Start Shopping' button |


*Figure 16:Cart Page - Items Added*

### 15.4.3 Checkout/Place Order

The checkout page handles two actions: selecting a delivery address and payment method to place an order, and adding a new delivery address if none exists yet.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /place_order |
| Address Dropdown | Populated from buyer's saved addresses in the Address table |
| No Address Warning | If no address exists, a red warning prompts the user to add one first |
| Payment Method | Dropdown: Cash on Delivery \| Credit Card \| Debit Card \| Bank Transfer \| JazzCash \| EasyPaisa |
| Place Order Button | Green success button — creates Order, OrderItem, Shipment, Payment records; clears CartItems |
| Add New Address Form | Second card on same page — Street, City, Country, Zip Code fields; POST to /add_address |
| Back Button | Returns to /cart without losing cart data |


### 15.4.4 My Orders

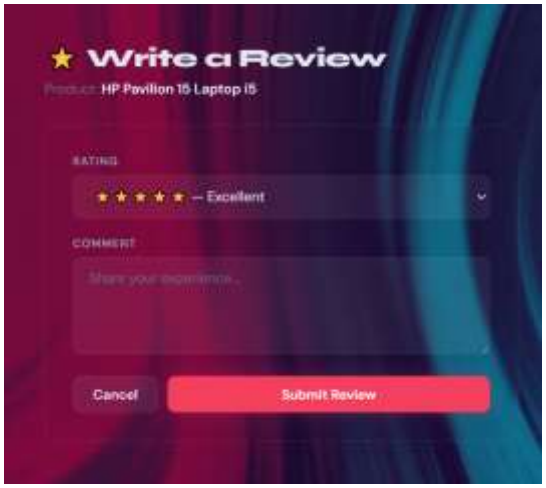
This page gives the buyer a complete history of all their past orders, with color-coded status badges for both order status and payment status.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /my_orders |
| Table Columns | Order ID \| Date \| Total Amount \| Order Status \| Payment Method \| Pay Status |
| Order Status Badges | delivered = green \| cancelled = red \| shipped = blue \| pending = yellow |
| Payment Status Badges | completed = green \| failed = red \| pending = yellow |
| Date Formatting | Displayed as DD Mon YYYY (e.g., 15 Jan 2025) using Jinja2 strftime |
| Empty State | Shows '🛍 No orders yet!' with 'Start Shopping' button |


### 15.4.5 Write Review

Buyers can leave a star rating and written comment for any product directly from the product grid's 'Review' button.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /review/<product_id> |
| Product Name | Displayed at top of form — fetched from DB by product_id |
| Rating Dropdown | 5 options — ☆ to ☆☆☆☆☆ with descriptive labels (Excellent to Terrible) |
| Comment Textarea | 4 rows — 'Share your experience...' placeholder |
| Submit Button | Red primary — inserts into Review table, redirects to buyer_home |
| Cancel Button | Returns to buyer_home without saving |




## 15.5 Seller Interface

The Seller interface provides a complete product management panel. Sellers can view their performance metrics, manage their product listings, add new products, edit existing ones, and perform soft-deletes. Sellers are redirected to /seller immediately after logging in.

### 15.5.1 Seller Dashboard (seller_dashboard.html)

The seller dashboard is the central hub for sellers, displaying key performance indicators at a glance and providing access to all product management actions.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /seller |
| Stats Row — Active Products | COUNT of products with status='active' for this seller |
| Stats Row — Total Revenue | SUM of (quantity × unit_price) from OrderItem JOIN Product — this seller's earnings |
| Stats Row — Total Listings | Total number of products listed by this seller (all statuses) |
| Products Table | Columns: ID \| Name \| Price \| Stock \| Status \| Actions |
| Status Badges | active = green \| inactive = yellow \| deleted = red |
| Edit Button | Blue info button — navigates to /seller/edit_product/<id> |
| Delete Button | Red danger button — soft-deletes (sets status='deleted') with JS confirm dialog |
| Add Product Button | Red primary button — top-right of table — links to /seller/add_product |
| Empty State | 'No products yet. Add your first one!' message |


## 15.5.2 Add Product (add_product.html)

Sellers can list new products through a clean form that assigns the product to a category/subcategory hierarchy fetched dynamically from the database.

| UI Element | Description / Functionality |
|---|---|
| **Page URL** | /seller/add_product |
| **Product Name** | Text input — required |
| **Price (Rs)** | Number input — step 0.01, min 1 — supports decimal prices |
| **Stock Quantity** | Number input — min 0 |
| **Category Dropdown** | Dynamically populated — shows 'Category > Subcategory' format (e.g., 'Electronics > Smartphones') |
| **Add Product Button** | Red primary — inserts into Product table with status='active', redirects to dashboard |
| **Cancel Button** | Returns to seller dashboard without saving |

## 15.5.3 Edit Product (edit_product.html)

Sellers can update any of their existing products. The form is pre-filled with the current product data fetched from the database, so the seller only needs to change the fields they want to update.

| UI Element | Description / Functionality |
|---|---|
| Page URL | /seller/edit_product/<product_id> |
| Pre-filled Fields | Name, Price, Stock — values pulled from Product table |
| Status Dropdown | active / inactive — sellers can deactivate a product without deleting it |
| Save Changes Button | Green success button — UPDATE query with ownership check (seller_id must match) |
| Cancel Button | Returns to seller dashboard without making changes |
| Security | Flask route checks seller_id = session user_id before updating — prevents cross-seller edits |

## 15.6 UI Design System

MarketHub uses a consistent design system defined entirely in base.html's embedded CSS. Every page inherits these design tokens, ensuring visual consistency across all 11 templates.

### 15.6.1 Color Palette

| UI Element | Description / Functionality |
|---|---|
| Primary Background | #F4F6F9 — light gray page background |
| Navbar / Dark Surfaces | #1A1A2E — deep navy blue |
| Accent / Primary Actions | #E94560 — red-pink for buttons, prices, active states |
| Accent Hover | #C73652 — darker red for hover states |
| Card Background | #FFFFFF — white cards with soft shadow |
| Body Text | #333333 — dark gray for readability |
| Muted Text | #888888 — gray for secondary/meta information |
| Success | #28A745 — green for success states and badges |
| Danger | #DC3545 — red for error states |
| Info | #17A2B8 — teal for informational states |

### 15.6.2 Reusable Components

| UI Element | Description / Functionality |
|---|---|
| .card | White rounded container (border-radius: 10px) with subtle box-shadow — used on all form and data pages |
| .btn / .btn-* | 6 button variants: primary (red), secondary (navy), success (green), danger (red), info (teal), sm (small size) |
| .product-card | Hover-lift effect (translateY -3px) — grid item for product listings |
| .stat-card | KPI display card — large colored number + label — used in Seller and Admin dashboards |
| .badge / .badge-* | Pill-shaped status indicator — 4 color variants matching alert colors |
| .form-group | Consistent label + input styling — red focus border, smooth transition |
| .search-bar | Flex row of input + select + buttons — used on buyer home |
| .stats-row | CSS Grid auto-fill — responsive stat card layout |
| .product-grid | CSS Grid auto-fill minmax(220px) — responsive product card layout |
| flash alerts | 4 color-coded alert boxes with left border accent — auto-displayed on every page |
| nav | Sticky top navbar — transparent logo + role-conditional links + logout button |
| footer | Centered copyright line — consistent across all pages |

### 15.6.3 Responsive Design

The layout uses CSS Grid with auto-fill and minmax() for the product grid and stats row, making them naturally responsive. Forms are constrained with max-width values (420px for login, 480px for register, 520px for product forms) and centered with margin: auto for consistent appearance on all screen sizes. The navbar uses flexbox with space-between for logo and links alignment.

## 15.7 User Flow Summary

| UI Element | Description / Functionality |
|---|---|
| Step 1 | / → redirects to /login |
| Step 2 | /login → enter credentials → Flask detects Buyer role → redirect to /buyer |
| Step 3 | /buyer → browse products, search/filter → click 'Add to Cart' |
| Step 4 | /cart → review items and total → click 'Checkout' |
| Step 5 | /place_order → add address (if needed) → select payment → click 'Place Order' |
| Step 6 | /my_orders → view order with status badge |
| Optional | /review/<id> → submit star rating and comment for a product |

| UI Element | Description / Functionality |
|---|---|
| **Step 1** | /login → Flask detects Seller role → redirect to /seller |
| **Step 2** | /seller → view stats (active products, revenue, total listings) |
| **Step 3** | /seller/add_product → fill form → product appears in dashboard |
| **Step 4** | /seller/edit_product/<id> → update name, price, stock, or status |
| **Step 5** | /seller → click Delete → JS confirm dialog → soft-delete (status='deleted') |

# 16. Reflection & Discussion

## 16.1 Assumptions Made During System Design

- A User can only be a Buyer or a Seller, not both simultaneously. This simplifies the ISA specialization but may not reflect real-world platforms like Amazon, where the same account can sell and buy.
- The Buyer_Order junction was included to support group/shared orders. In practice, most e-commerce platforms maintain a strict 1:M Buyer-to-Order relationship. This junction was included to fulfill the requirement of at least two M:N relationships.
- product_id in the Product table refers to the item type, not individual physical units. Stock quantity is managed at the catalog level, not per-unit tracking.
- Payment.amount is assumed to equal Order.total_amount at checkout. Partial payments, installments, and discount codes are not modeled in the current iteration.
- Admin functionality was removed from the ER design due to scope constraints. It can be implemented as a role/permission flag on the User entity in a future version.
- All monetary values are stored in PKR (Pakistani Rupees). Multi-currency support would require an additional Currency table and exchange rate management.

## 16.2 Challenges Encountered

- Resolving Multiple M:N Relationships: Identifying and correctly modeling four many-to-many relationships (Cart↔Product, Order↔Product, Product↔Warehouse, Buyer↔Order) required careful attention to ensure each junction table was properly normalized and did not introduce redundancy.
- Cascade Delete vs Restrict Tradeoffs: Deciding where to use ON DELETE CASCADE required careful analysis. For critical tables like Order and Product, cascade deletion was avoided to protect historical records. For weak entities (ProductImage, CartItem), cascade deletion was appropriate.
- Derived Attributes in T-SQL: Implementing derived attributes (total_amount, Seller.rating) required user-defined functions and views rather than stored columns, to avoid data redundancy and stale values.
- Composite Primary Keys with Partial Dependencies: Ensuring 2NF and 3NF for CartItem and OrderItem required verifying that every non-key attribute depended on the full composite PK, not just one part of it.
- T-SQL Reserved Keywords: The table name 'Order' is a T-SQL reserved keyword and must always be enclosed in square brackets ([Order]) in all SQL statements.

## 16.3 Limitations of the Current Design

- No Discount / Coupon System: Promotions, discount codes, and tiered pricing are not modeled. Adding this would require a Coupon entity with M:N links to Product and Order.
- Single-Currency Support: All prices are in one currency. International marketplace support would require a Currency table.
- No Notification System: Real-world platforms require order confirmation emails, shipping updates, and stock alerts. These would require an external messaging/event system.
- Product Variants Not Fully Modeled: While ProductAttribute allows dynamic key-value specs, a true variant system (e.g., Size M / Color Red as a separate SKU) would require a dedicated ProductVariant entity with its own stock tracking.
- Review Verification: The business rule that a buyer may only review a product they purchased is enforced at the application level, not by a DB constraint. A CHECK constraint cannot join tables; a trigger would be more robust.
- No Full-Text Search: Product browsing uses LIKE '%keyword%' which is slow on large datasets. A real system would require a full-text index or a dedicated search service (e.g., Elasticsearch).

## 16.4 Potential Improvements for Real-World Deployment

- Add a ProductVariant table to properly handle SKUs with multiple attributes (color, size, etc.) and variant-level stock tracking.
- Implement a full-text search index on Product.name and Product description for fast catalog search.
- Add a CouponCode and Promotion table to support discount management.
- Replace the boolean is_default on Address with a separate UserDefaultAddress table with a UNIQUE constraint to eliminate potential race conditions in concurrent updates.
- Add audit logging tables to track changes to critical entities (Product price changes, Order status transitions).
- Consider row-level security (RLS) in SQL Server to restrict Sellers to seeing only their own product and order data when accessing the database directly.
- Implement database partitioning on the Order and Transaction tables by order_date for scalability as data volume grows.

# 17. Requirements Compliance Checklist

All minimum complexity requirements defined in the EC-240 CEP specification have been met or exceeded.

| Requirement | Status | Evidence |
|---|---|---|
| **At least 6–8 entities** | **Exceeds** | 21 entities implemented across all subsystems |
| **At least 2 M:N relationships** | **Exceeds** | 4 M:N: Cart↔Product, Order↔Product, Product↔Warehouse, Buyer↔Order |
| **Weak entities or specialization** | **Done** | ISA: Buyer/Seller; Weak: ProductImage, ProductAttribute, CartItem, OrderItem |
| **Normalization to 3NF** | **Done** | All 21 tables analyzed and confirmed in 3NF (see Section 7) |
| **At least 15 SQL queries** | **Exceeds** | 17+ queries including analytics, trend, inactive, aggregated, advanced |
| **JOINs (multi-table)** | **Done** | Queries 11 (5-table JOIN), 12 (4-table JOIN), 13 (correlated subquery) |
| **Nested subqueries** | **Done** | Queries 13 and 14 use correlated and scalar subqueries |
| **GROUP BY & HAVING** | **Done** | Queries 15, 16 - HAVING filters on aggregated conditions |
| **Complex logical conditions** | **Done** | Query 17 - multi-condition WHERE with AND and threshold comparison |
| **Indexes** | **Done** | 6 indexes: idx_product_seller_id, idx_order_buyer_id, idx_review_buyer_product, etc. |
| **Views** | **Done** | 3 views: ProductCatalog, OrderSummary, SellerDashboard |
| **Triggers** | **Done** | 3 triggers: trg_reduce_stock, trg_prevent_negative_stock, trg_update_cart_timestamp |
| **Additional constraints** | **Done** | CHECK, UNIQUE, DEFAULT, CASCADE DELETE across all tables |
| **User-Defined Functions** | **Done** | fn_CalculateOrderTotal, fn_SellerRating |
| **Transactions (TRY-CATCH)** | **Done** | Full order-placement transaction with ROLLBACK on error |
| **Frontend Interface** | **Done** | Specifications defined in Section 15; implementation in progress |
| **Project Report** | **Done** | This document covers all required sections |

# 18. Team Division of Work

| Member | Role | Responsibilities |
|---|---|---|
| Mawa Chaudhary (474121) | ER Design & Requirements Lead | Defined system scope and functional requirements; designed ER diagram (entities, relationships, specialization, weak entities, cardinality constraints); documented business rules; prepared report structure and final documentation. |
| Muhammad Hassan Ali (456543) | Database Schema & Normalization | Transformed ER model into 21 relational tables; defined all primary keys, foreign keys, and integrity constraints; performed 3NF normalization analysis for all tables; wrote all CREATE TABLE SQL scripts. |
| Rida Zahra (465791) | SQL Implementation & Analytics | Inserted realistic sample dataset; implemented all CRUD SQL operations; wrote analytical queries (top buyers, products, revenue trends); implemented advanced queries (JOINs, subqueries, GROUP BY, HAVING); generated business insights. |
| Tazeen ur Rehman (466342) | Frontend & System Enhancements | Developed user interface (HTML/CSS/JS); connected frontend to database backend; implemented indexes, views, triggers, and user-defined functions; designed analytics dashboard display. |

# 19. Appendix

1. Lucid Link of ER Diagram :
   https://lucid.app/lucidchart/803fc2d3-2102-45de-8bd9-4c41c83313d8/edit?viewport_loc=-824%2C8%2C3140%2C1334%2Cp1&invitationId=inv_162cc041-84ac-4155-be6f-01bad0fba4a5

2. Draw.io Link Relational Schema:
   https://drive.google.com/file/d/1iccTfl_kIUryGEDOXpK15JyZLIzxxPWP/view?usp=sharing

3. SQL Queries uploaded on GitHub:
   https://github.com/Rzrida/MarketHub-Online-Marketplace-System---SQL-Queries

4. Flask Python code uploaded on Github:
   https://github.com/Tazeen-R/MarketHubFrontend